



UNIVERSIDADE
FEDERAL DO CEARÁ

Relatório Final

Manutenção De Software - 01 - 2026.1

Profa. Carla Ilane Moreira Bezerra

Autores

Marcio Alexandre Martins Almeida Junior - 536040

Francisco Rafael Lobo Pinho - 542173

Sumário

Sumário	1
1. Caracterização do Projeto	3
2. Métricas Antes da Refatoração	3
2.1 Radon Antes da Refatoração	3
2.1.1 Complexidade Ciclométrica por Arquivo Antes da Refatoração	3
2.1.2 Complexidade Ciclométrica por Função Antes da Refatoração	4
2.1.3 Índice de Manutenibilidade Antes da Refatoração	4
2.1.4 Métricas brutas Antes da Refatoração	5
2.2 CodeCarbon Antes da Refatoração	5
2.3 Pylint Antes da Refatoração	5
2.3.1 Distribuição do Smells Antes da Refatoração	5
2.3.2 Distribuição dos Problemas de Código Antes da Refatoração	6
2.3.3 Arquivos mais Críticos Antes da Refatoração	6
2.3.4 Score Projeto Antes da Refatoração	7
2.4 Pytest Antes da Refatoração	7
2.4.1 Cobertura de testes Antes da Refatoração	7
2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração	9
3. Processo de Refatoração	9
3.1 too-many-statements	9
3.1.1 Ocorrência Número 1	9
3.1.2 Ocorrência Número 2	10
3.2 too-many-instance-attributes	11
3.3 too-many-branches	11
4. Métricas Após a Refatoração	11
4.1 Radon Após a Refatoração	11
4.1.1 Complexidade Ciclométrica por Arquivo Após a Refatoração	11
4.1.2 Complexidade Ciclométrica por Função Após a Refatoração	11
4.1.3 Índice de Manutenibilidade Após a Refatoração	12
4.1.4 Métricas Brutas Após a Refatoração	12
4.2 CodeCarbon Após a Refatoração	12
4.3 Pylint Após a Refatoração	13
4.3.1 Distribuição do Smells Após a Refatoração	13
4.3.2 Distribuição dos Problemas de Código Após a Refatoração	13
4.3.3 Arquivos mais Críticos Após a Refatoração	13
4.3.4 Score Projeto Após a Refatoração	13
4.4 Pytest Após a Refatoração	13
4.4.1 Cobertura de testes Após a Refatoração	13
4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração	13

5. Comparação dos Resultados	14
6. Percepções sobre a Prática	14
7. Links	14

1. Caracterização do Projeto

Esta seção deve conter: nome, descrição e propósito do projeto open source escolhido. Confira um exemplo abaixo:

Nome Projeto: Advanced Alchemy

Advanced Alchemy é uma biblioteca Python de código aberto que funciona como uma camada complementar ao SQLAlchemy, fornecendo recursos prontos para uso em aplicações que precisam persistir dados em bancos relacionais. As principais capacidades incluem:

- * Repositórios síncronos e assíncronos com operações CRUD comuns e operações em lote (bulk insert, update, upsert e delete) otimizadas por dialeto de banco;
- * Integração nativa com os principais frameworks web Python, como Litestar, Starlette, FastAPI e Sanic;
- * Classes base utilitárias com colunas de auditoria, chaves primárias pré-configuradas (UUID ou Big Integer) e suporte a chaves primárias compostas;
- * Tipos de dados customizados, incluindo armazenamento de arquivos (File Object) com múltiplos backends, texto criptografado, hash de senhas (Argon2, Passlib, Pwplib), segredos TOTP e códigos de verificação com expiração e bloqueio por tentativas;
- * Suporte testado a múltiplos bancos de dados, como SQLite, PostgreSQL, MySQL, Oracle, SQL Server, CockroachDB, DuckDB e Google Spanner;
- * Roteamento de réplicas de leitura/escrita e integração com cache de consultas via Dogpile.

O propósito do Advanced Alchemy é reduzir o trabalho repetitivo que desenvolvedores enfrentam ao usar SQLAlchemy puro, oferecendo uma camada de repositórios e serviços já pronta, testada e otimizada para tarefas comuns de persistência de dados. O projeto resolve um problema prático recorrente: implementar manualmente operações de CRUD, paginação, filtros, auditoria e

suporte a múltiplos bancos de dados para cada novo projeto é trabalhoso e propenso a erros. Ao fornecer essas soluções de forma padronizada e mantida ativamente, o Advanced Alchemy permite que equipes integrem persistência de dados de forma mais rápida e consistente em frameworks web populares do ecossistema Python.

2. Métricas Antes da Refatoração

Nesta seção você vai apresentar as principais métricas obtidas antes da refatoração para cada ferramenta, não precisa ser todas as métricas. Confira um exemplo abaixo:

2.1 Radon Antes da Refatoração

2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela antes da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
advanced_alchemy/operations.py	8	10.12	35	81	E	create_merge_upsert
advanced_alchemy/_listeners.py	36	4.28	33	154	E	handle_multiple_attribute
advanced_alchemy/exceptions.py	6	6.83	27	41	D	wrap_sqlalchemy_exception
advanced_alchemy/extensions/litestar/dto.py	14	6.43	27	90	D	generate_field_definitions
advanced_alchemy/extensions/litestar/providers.py	25	3.88	24	97	D	provide_filters

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
advanced_alchemy/__main__.py	1	1.0	1	1	A	run_cli
advanced_alchemy/cache/config.py	1	1.0	1	1	A	_default_arguments
advanced_alchemy/extensions/flask/utlis.py	1	1.0	1	1	A	jsonify

advanced_alchemy/extensions/litestar/plugins/init/config/engine.py	1	1.0	1	1	A	serializer
advanced_alchemy/extensions/flask/cli.py	2	1.0	1	2	A	get_database_migration_plugin

2.1.2 Complexidade Ciclomática por Função Antes da Refatoração

Apresente as funções do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela antes da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
advanced_alchemy/operations.py	method	create_merg_upsert	E	35
advanced_alchemy/_listeners.py	method	handle_multiple_attribute	E	33
advanced_alchemy/exceptions.py	function	wrap_sqlalchemy_exception	D	27
advanced_alchemy/extensions/litestar/dto.py	method	generate_field_definitions	D	27
advanced_alchemy/extensions/litestar/providers.py	function	provide_filters	D	24

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
advanced_alchemy/__main__.py	function	run_cli	A	1
advanced_alchemy/_listeners.py	function	set_async_context	A	1
advanced_alchemy/_listeners.py	function	reset_async_context	A	1
advanced_alchemy/_listeners.py	function	is_async_context	A	1
advanced_alchemy/_listeners.py	method	_is_listener_enabled	A	1

2.1.3 Índice de Manutenibilidade Antes da Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela antes da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	mi	rank_mi
---------	----	---------

advanced_alchemy/repository/_async.py	0.0	C
advanced_alchemy/repository/_sync.py	0.0	C
advanced_alchemy/utils/serialization.py	21.0395	A
advanced_alchemy/repository/memory/_sync.py	22.8584	A
advanced_alchemy/_listeners.py	27.7398	A

Tabela de melhores métricas

arquivo	mi	rank_mi
advanced_alchemy/__init__.py	100.0	A
advanced_alchemy/__main__.py	100.0	A
advanced_alchemy/__metadata__.py	100.0	A
advanced_alchemy/cache/_null.py	100.0	A
advanced_alchemy/cache/config.py	100.0	A

2.1.4 Métricas brutas Antes da Refatoração

Essas métricas podem ser encontradas no arquivo *raw_por_arquivo_e_total_antes.csv* Traga apenas o total, que se encontra no final do csv.

loc total	lloc total	sloc total	comments total	multi
38511	15213	22706	1634	8554

2.2 CodeCarbon Antes da Refatoração

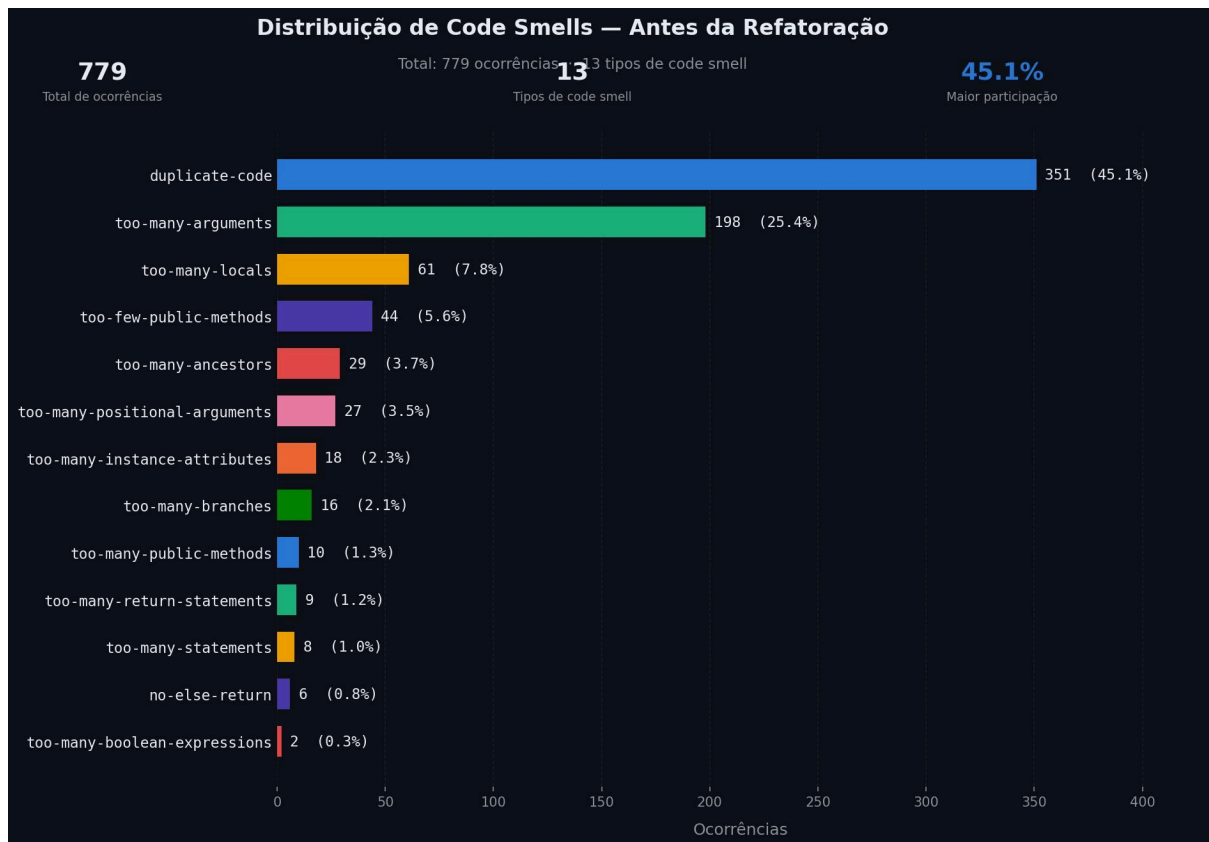
Preencha a tabela abaixo com as métricas ambientais antes da refatoração.

duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
0.0371	2.6077966909505312e-08	7.025199482748555e-07	1.6355707529308594e-07	1.0160303888824273e-07	2.6516011418132865e-07

2.3 Pylint Antes da Refatoração

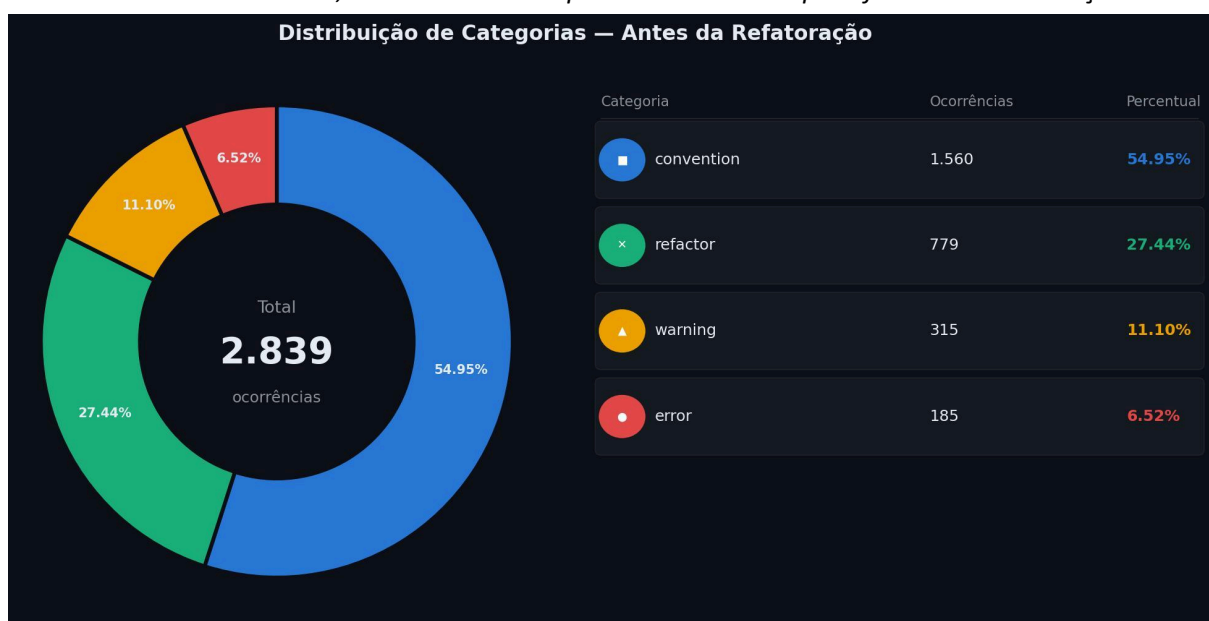
2.3.1 Distribuição do Smells Antes da Refatoração

A partir do JSON *pylint_ranking_smells_antes.json* gere um gráfico para representar a distribuição dos code smells antes da refatoração. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.



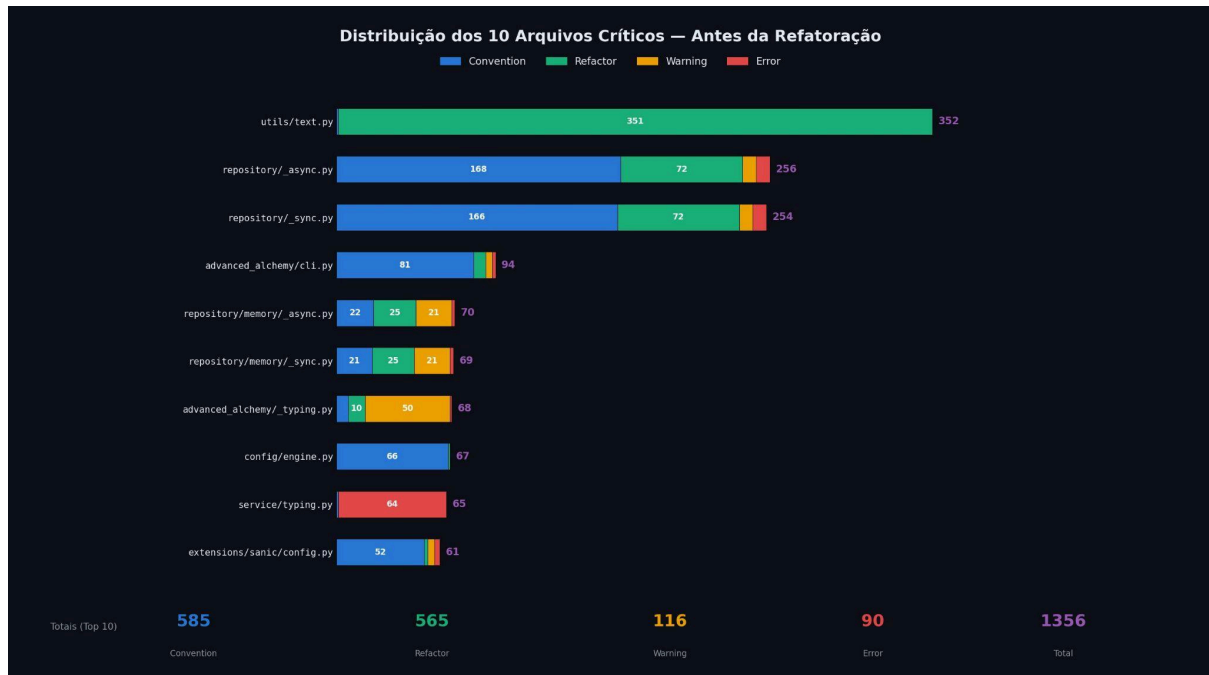
2.3.2 Distribuição dos Problemas de Código Antes da Refatoração

A partir do JSON `pylint_distribuicao_categorias_antes.json` gere um gráfico para representar a distribuição entre as 4 categorias de problemas de código antes da refatoração. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.



2.3.3 Arquivos mais Críticos Antes da Refatoração

A partir do JSON `pylint_arquivos_criticos_antes.json` gere um gráfico para representar a distribuição dos 10 arquivos mais críticos antes da refatoração. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.



2.3.4 Score Projeto Antes da Refatoração

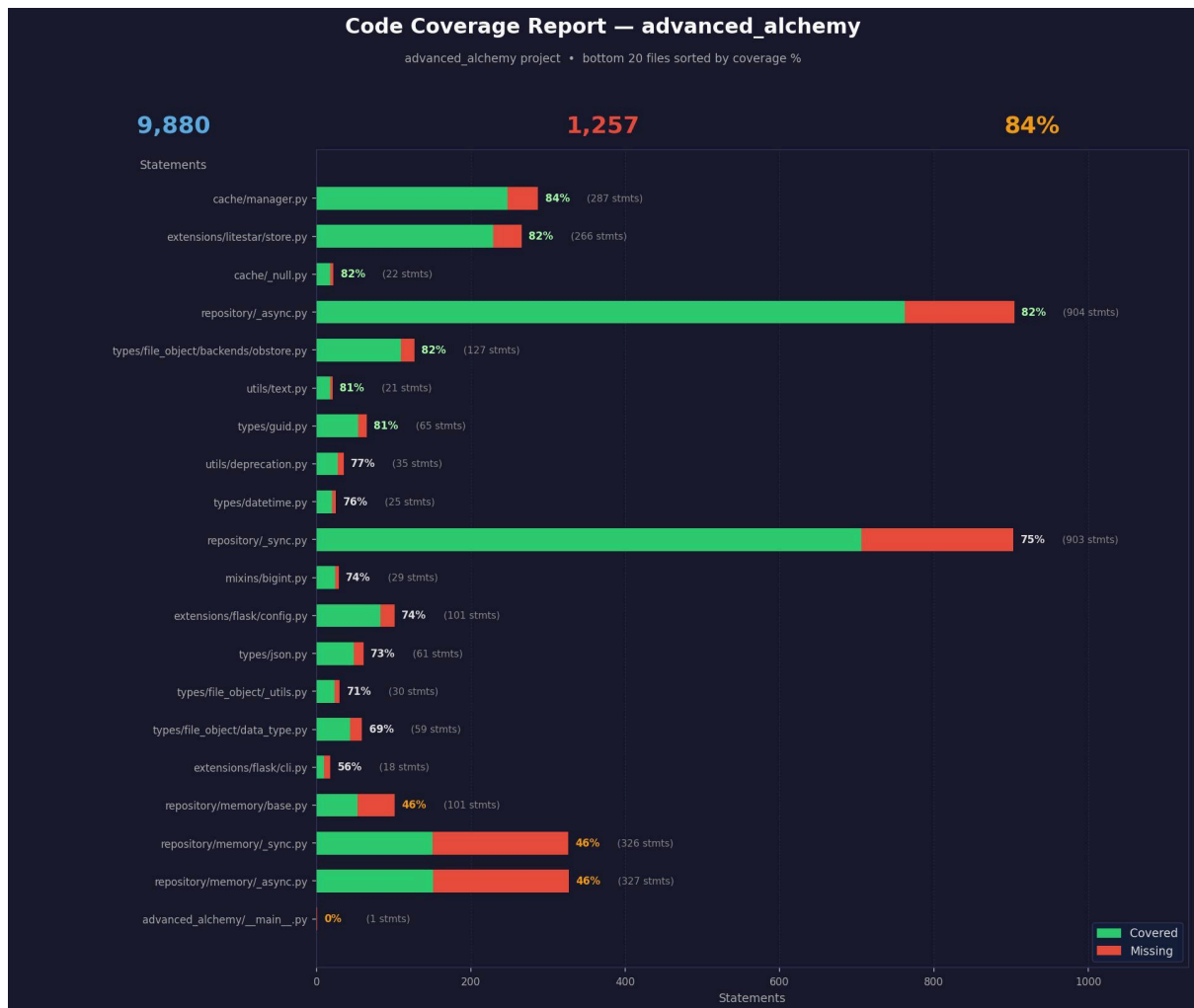
A partir do text `pylint_score_antes.txt` informe o score que o pylint deu ao projeto antes da refatoração. Não precisa ser uma imagem, fica a critério de vocês.



2.4 Pytest Antes da Refatoração

2.4.1 Cobertura de testes Antes da Refatoração

Gere um gráfico da cobertura de testes do projeto antes da refatoração. Para isso, basta visualizar a cobertura de testes na página HTML, copiar os dados da tabela e gerar o gráfico com o auxílio de uma LLM. Para visualizar a página de cobertura de testes antes da refatoração, execute o seguinte comando: `start "metrics-before-pytest\coverage_antes_html\index.html"`. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.



2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração

Aqui você deve informar quantos testes falharam e quantos testes passaram antes da refatoração. Para isso, basta visualizar a página HTML `pytest_antes.html`. Para acessá-la, execute o seguinte comando: `start "metrics-before-pytest\pytest_antes.html"`.

Quantidade de testes que passaram: 2706

Quantidade de testes que falharam: 0

3. Processo de Refatoração

Aqui você deve registrar o processo de refatoração dos 3 tipos de smells e de todas as ocorrências referentes a esses três tipos. Antes de iniciar a descrição de cada smell, informe qual LLM foi utilizada para realizar a refatoração.

Modelo utilizado: Claude Sonnet 4.6

Modelo utilizado: GPT-5.4 mini (plano free)

3.1 too-many-statements

Primeiramente, informe a quantidade de ocorrências desse tipo de smell.

Quantidade de ocorrências: 8

3.1.1 Ocorrência Número 1

Você deve preencher a tabela de identificação da ocorrência.

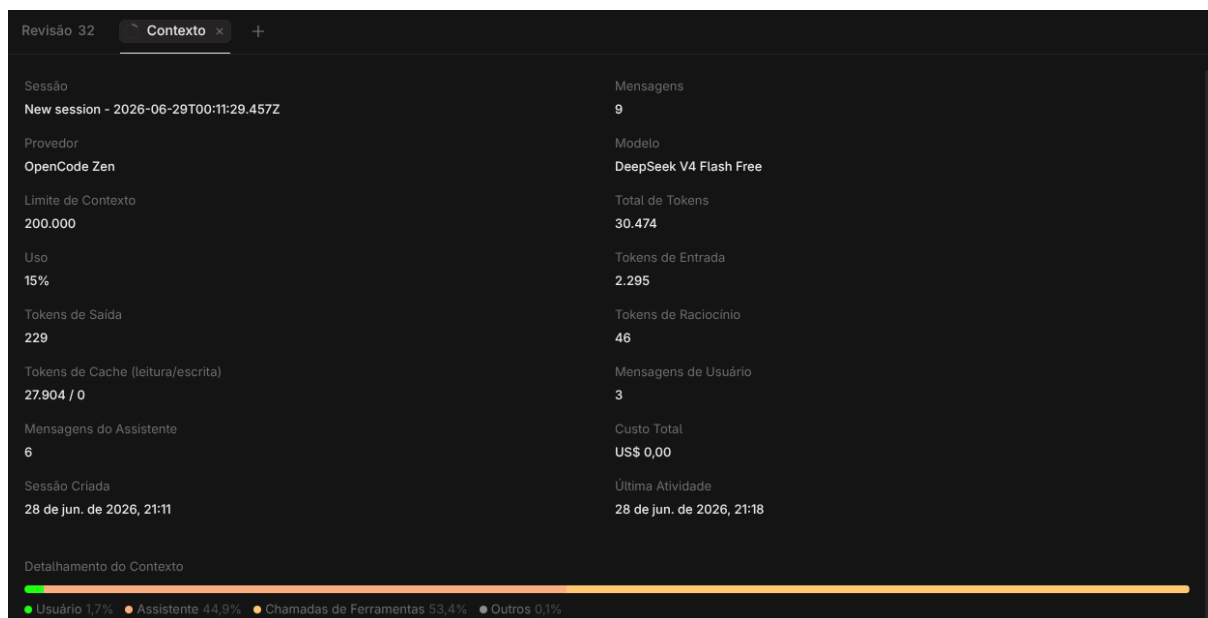
Tabela de identificação

module	obj	message
"advanced_alchemy._listeners"	"FileObjectInspector.handle_multiple_attribute"	"Too many statements (55/50)"

Seguiu o plano de refatoração: Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Link da publicação OpenCode: <https://opncd.ai/share/yfSNBCyY>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função `run` tinha lógica complexa e sensível a erros, então dividir o código em helpers reduziu a complexidade e melhorou a manutenção sem alterar o comportamento. Por outro lado, o uso da LLM consumiu mais tempo e tokens do que uma correção trivial exigiria, então o custo foi moderado. Ainda assim, neste caso específico, o ganho em segurança e clareza compensou o esforço.

3.1.2 Ocorrência Número 2

Você deve preencher a tabela de identificação da ocorrência.

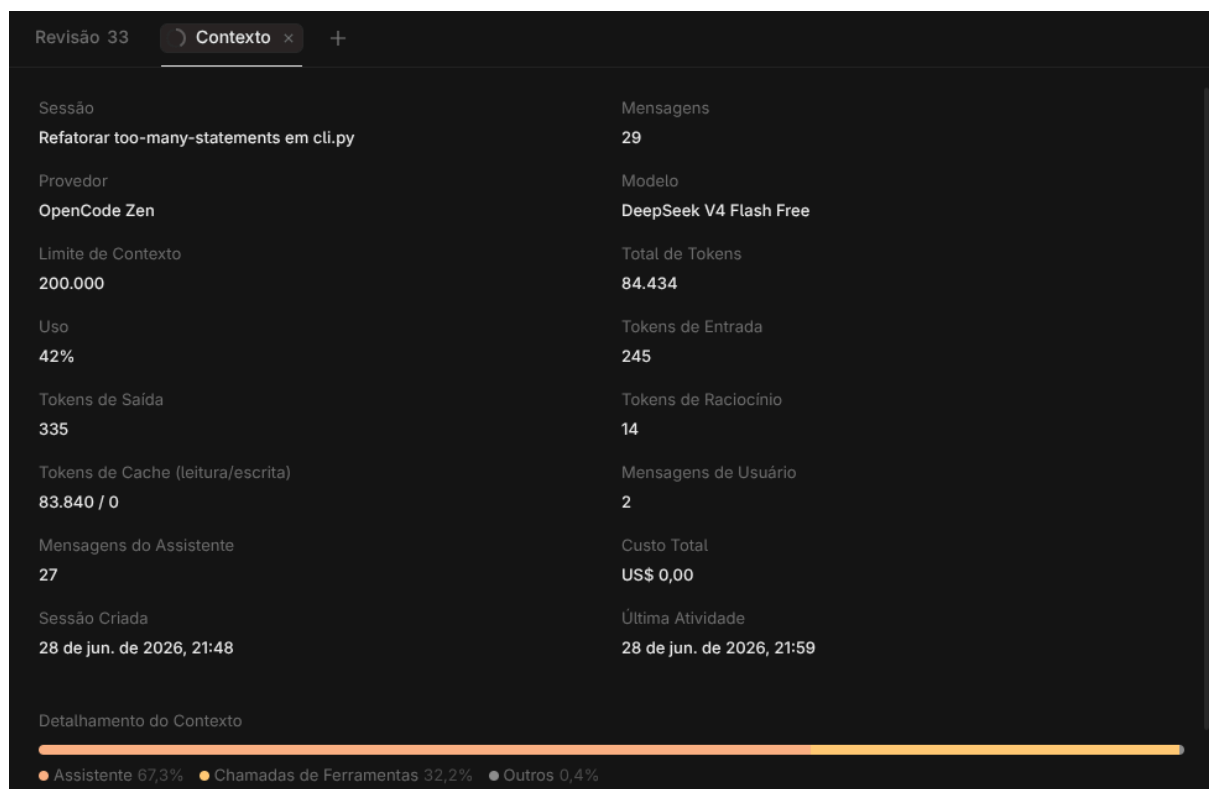
Tabela de identificação

module	obj	message
"advanced_alchemy.cli"	add_migration_commands"	"Too many statements (154/50)"

Seguiu o plano de refatoração: Sim, o plano de refatoração sugerido resolvia corretamente a ocorrência do smell, conforme a técnica de refatoração do Pylint, além de não causar o surgimento de smells críticos nem prejudicar o comportamento esperado do sistema. (Este é apenas um texto de exemplo.)

Link da publicação OpenCode: <https://opnkd.ai/share/RC7a1Brt>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função `add\_migration\_commands` reunia muitas responsabilidades em um único lugar, o que deixava o código grande e difícil de entender. Ao separar os comandos, opções compartilhadas e funções auxiliares, a função principal fica muito mais simples, sem alterar seu comportamento. Apesar de exigir mudanças em vários trechos do arquivo, o resultado é um código mais organizado, fácil de manter e que atende ao limite de complexidade definido pelo Pylint.

3.1.2 Ocorrência Número 3

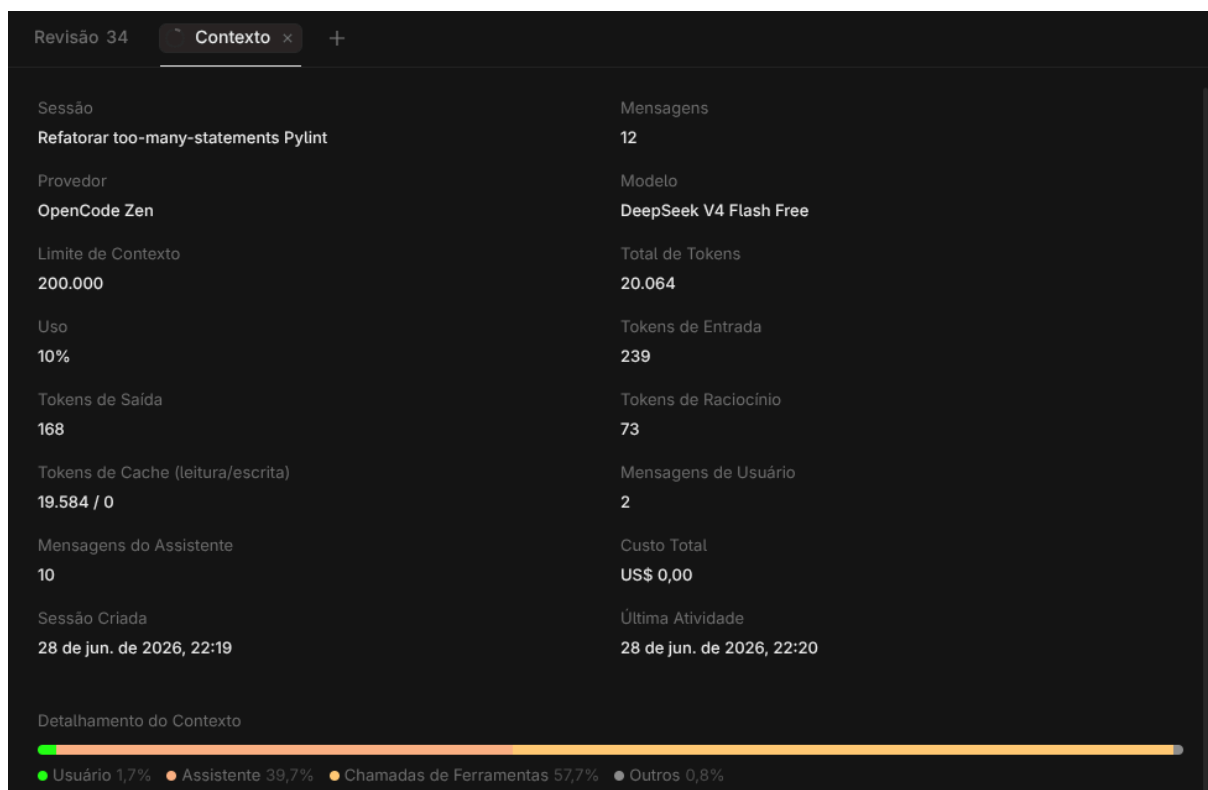
Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

module	obj	message
advanced_alchemy.exceptions	"wrap_sqlalchemy_exception"	"Too many statements (59/50)"

Link da publicação OpenCode: <https://opncd.ai/share/SFFAZNDH>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função `wrap_sqlalchemy_exception` repetia o mesmo padrão de tratamento em vários blocos `except`, deixando o código maior e mais difícil de manter. Ao extrair essa lógica para funções auxiliares, a função principal fica mais enxuta e focada apenas em direcionar cada tipo de exceção, sem alterar seu comportamento. Embora tenha sido necessário criar alguns helpers, o ganho em organização, reutilização e conformidade com o Pylint justifica o esforço.

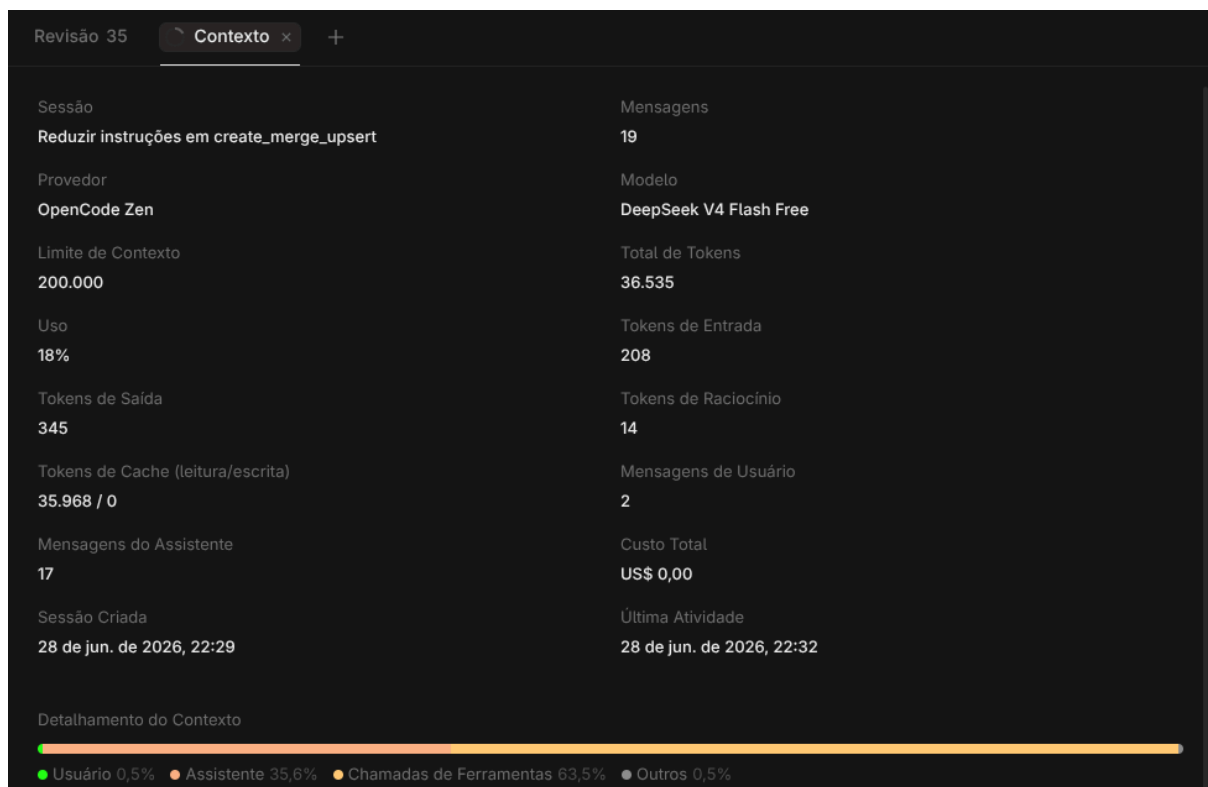
3.1.2 Ocorrência Número 4

Tabela de identificação

module	obj	message
<code>advanced_alchemy.operations</code>	<code>OnConflictUpsert.create_merge_upsert</code>	"Too many statements (64/50)",

Link da publicação OpenCode: <https://opncd.ai/share/cKERd5f2>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque o método `create_merge_upsert` tinha muita lógica concentrada em um único lugar. Separar essa lógica em métodos auxiliares deixou o código mais organizado e fácil de manter, sem alterar seu comportamento.

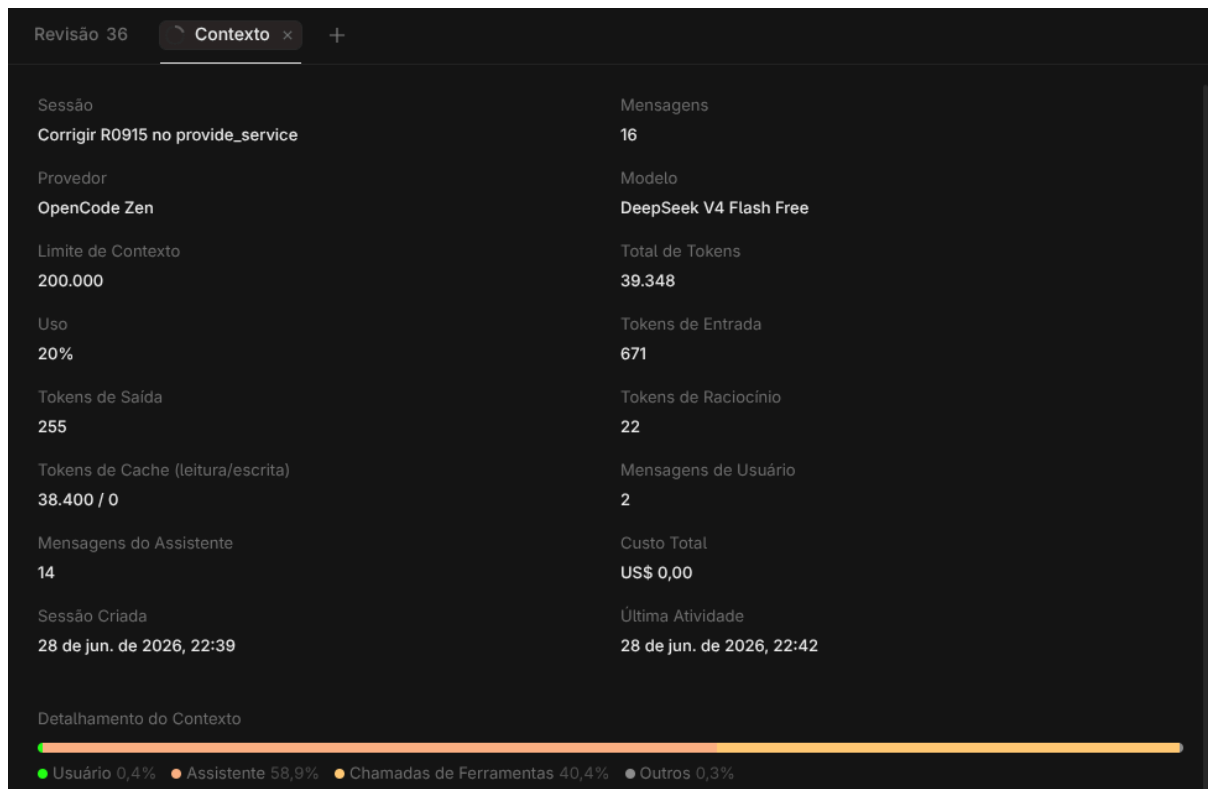
3.1.2 Ocorrência Número 5

Tabela de identificação

module	obj	message
"advanced_alchemy.extensions.fastapi.providers",	"provide_service"	"Too many statements (68/50)",

Link da publicação OpenCode: <https://opncd.ai/share/hr1sx54x>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função `provide_service` repetia a mesma lógica de limpeza de sessão em dois lugares. Ao mover essa lógica para funções auxiliares, o código ficou menor, mais organizado e fácil de manter, sem mudar seu comportamento.

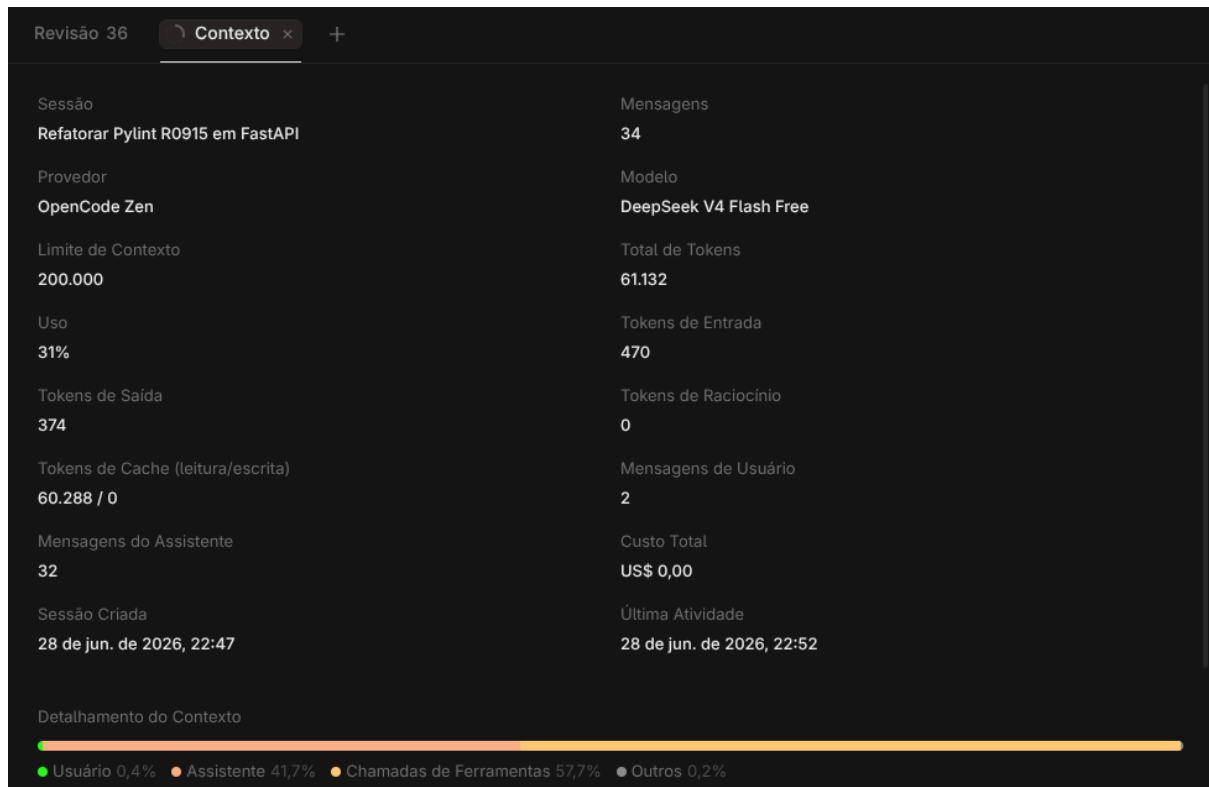
3.1.2 Ocorrência Número 6

Tabela de identificação

module	obj	message
"advanced_alchemy.extensions.fastapi.providers"	"_create_filter_aggregate_function_fastapi",	"Too many statements (74/50)",

Link da publicação OpenCode: <https://opncd.ai/share/jqkXMWeL>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função `_create_filter_aggregate_function_fastapi` concentrava a criação de vários providers em um único lugar. Ao separar cada provider em funções auxiliares, o código ficou mais organizado, fácil de manter e abaixo do limite do Pylint, sem alterar seu funcionamento.

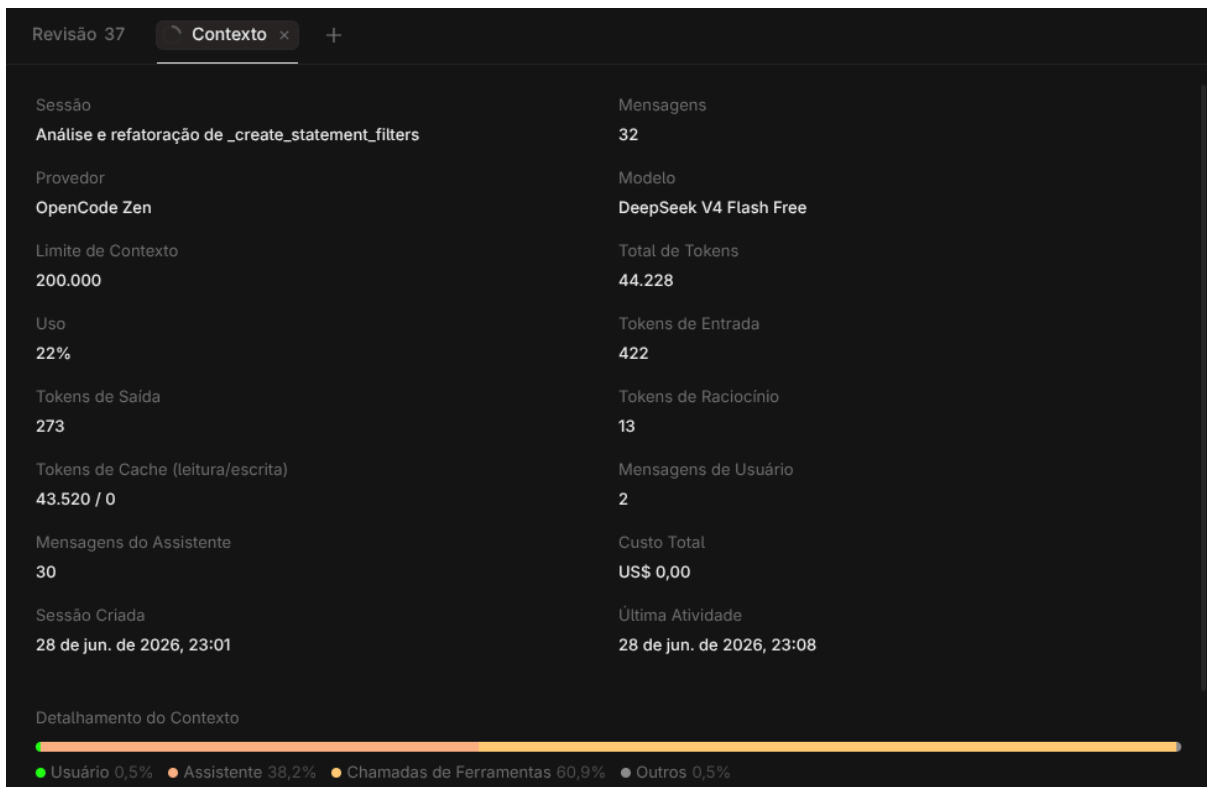
3.1.2 Ocorrência Número 7

Tabela de identificação

module	obj	message
advanced_alchemy.extensions. litterstar.providers",	"_create_statement_filters",	"Too many statements (57/50)",

Link da publicação OpenCode: <https://opncd.ai/share/PBihauZo>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função `_create_statement_filters` concentrava muita lógica para criar diferentes providers de filtro. Ao mover os blocos mais complexos para funções auxiliares, o código ficou mais organizado, fácil de manter e dentro do limite do Pylint, sem mudar seu comportamento.

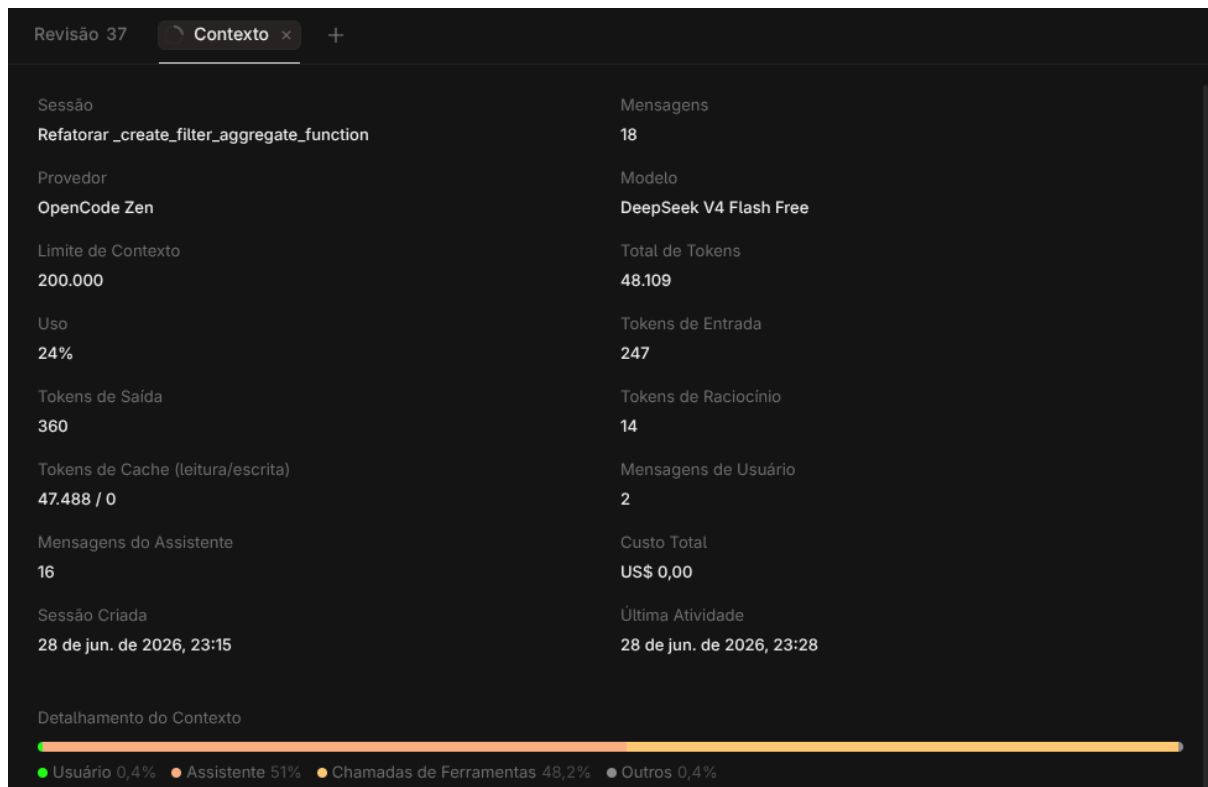
3.1.2 Ocorrência Número 8

Tabela de identificação

module	obj	message
"advanced_alchemy.extensions .littestar.providers",	"_create_filter_aggregate_function",	Too many statements (69/50)",

Link da publicação OpenCode: <https://opncd.ai/share/uMkyKSO9>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a função `\_create\_filter\_aggregate\_function` concentrava muita lógica repetitiva em um único lugar. Ao extrair essa lógica para funções auxiliares, o código ficou mais limpo, fácil de manter e dentro do limite do Pylint, sem alterar seu comportamento.

3.2 too-many-instance-attributes

3.2.1 Ocorrência Número 1

Primeiramente, informe a quantidade de ocorrências desse tipo de smell.

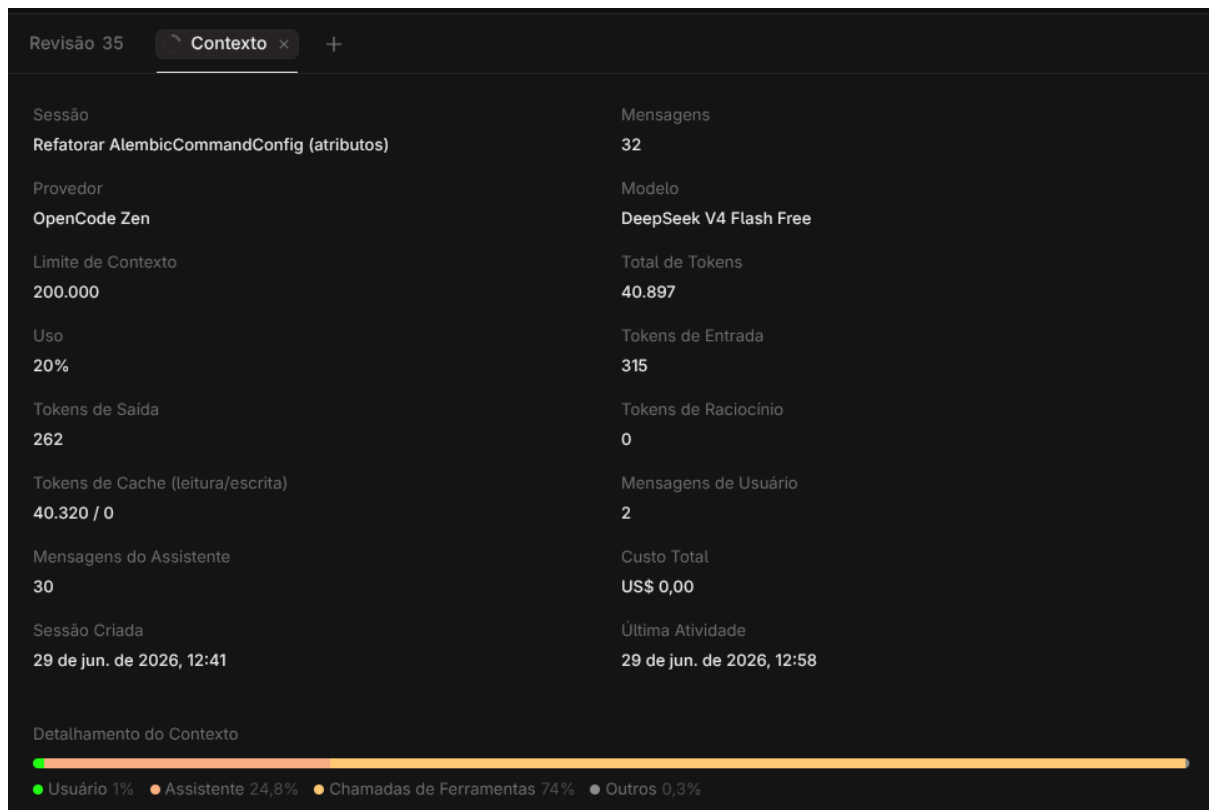
Quantidade de ocorrências: 18

Tabela de identificação

module	obj	message
"advanced_alchemy.alembic.commands",	"AlembicCommandConfig",,	"Too many instance attributes (10/7)"

Link da publicação OpenCode: <https://opncd.ai/share/oKnRJlil>

Print consumo de token:



Análise da refatoração:A refatoração valeu a pena porque a classe AlembicCommandConfig acumulava muitos atributos relacionados. Agrupá-los em dataclasses deixou a estrutura mais organizada e reduziu o número de atributos, atendendo ao limite do Pylint sem mudar o comportamento da classe.

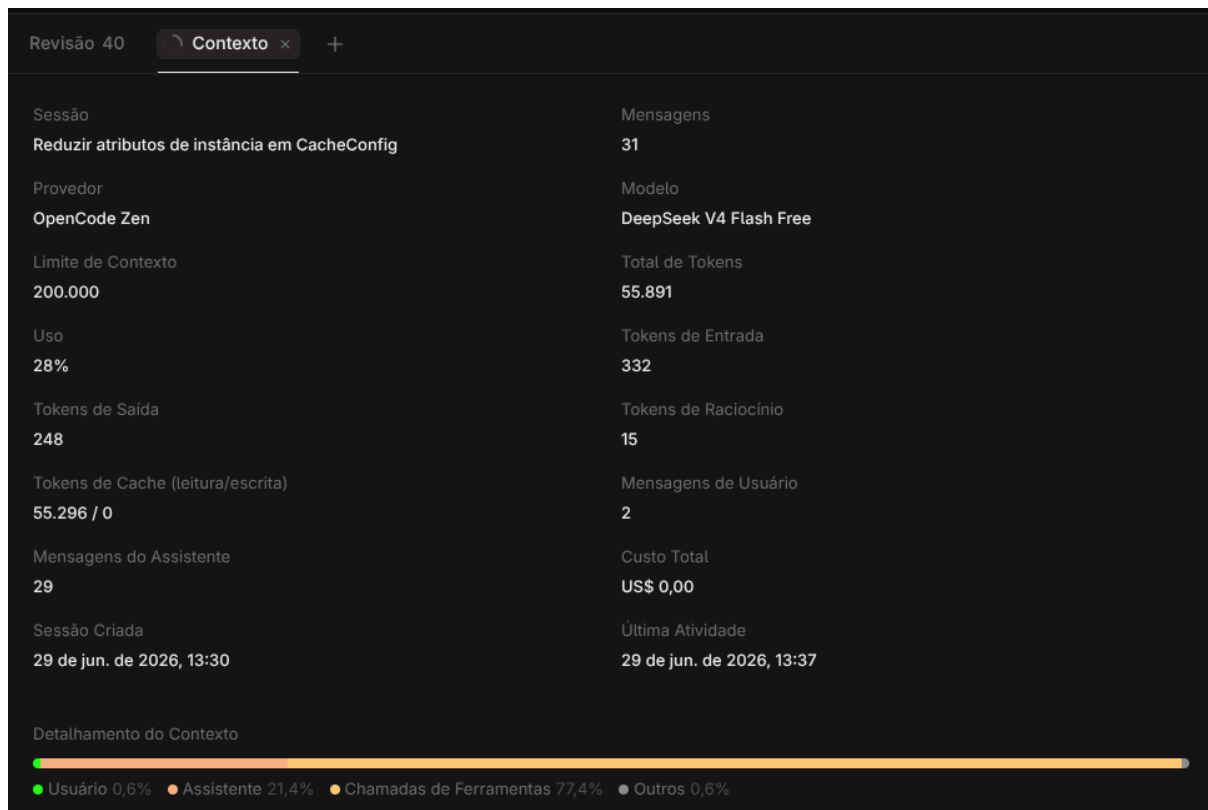
3.2.1 Ocorrência Número 2

Tabela de identificação

module	obj	message
"advanced_alchemy.cache.config",	CacheConfig",	"Too many instance attributes (8/7)",

Link da publicação OpenCode:<https://opncd.ai/share/sLWsURqX>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a classe `CacheConfig` tinha atributos de serialização que sempre eram usados em conjunto. Agrupá-los em um dataclass deixou a configuração mais organizada e reduziu o número de atributos, atendendo ao limite do Pylint sem alterar o comportamento da classe.

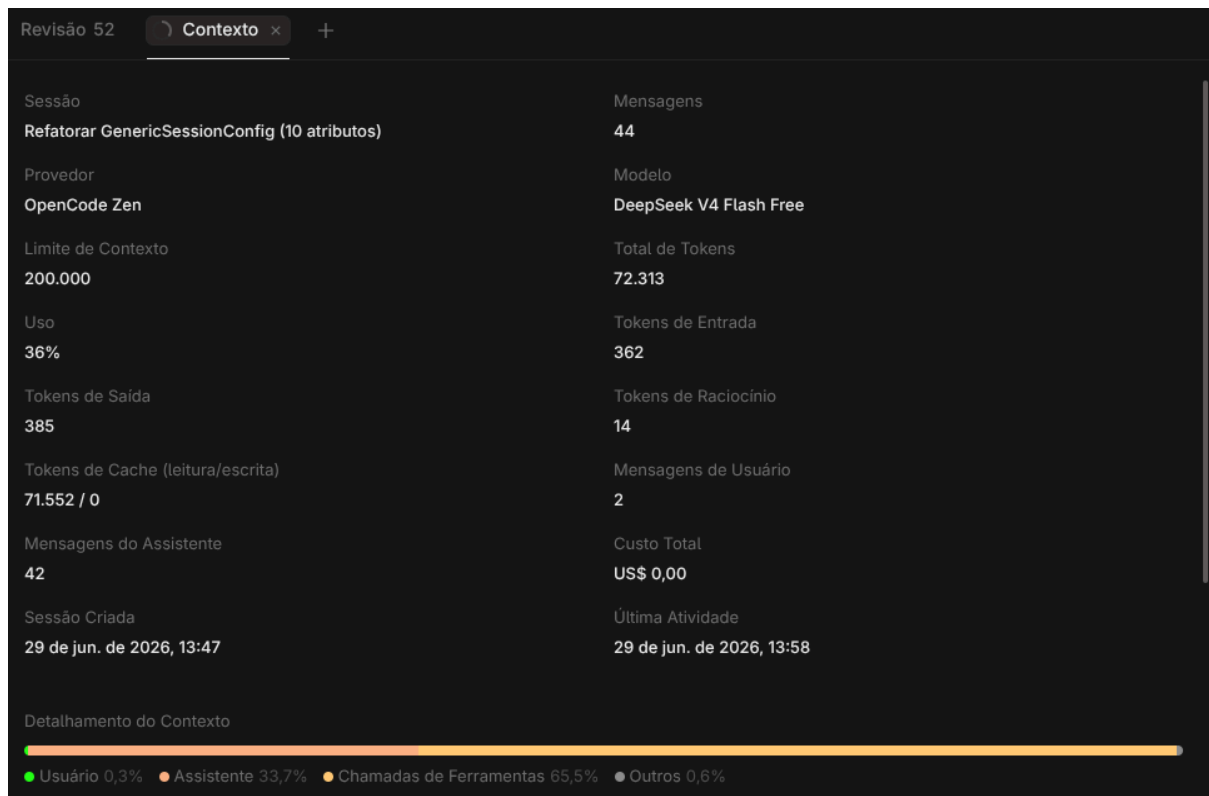
3.2.1 Ocorrência Número 3

Tabela de identificação

module	obj	message
"advanced_alchemy.config.common"	"GenericSessionConfig"	"Too many instance attributes (10/7)"

Link da publicação OpenCode: <https://opncd.ai/share/HuCNT3ar>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a classe `GenericSessionConfig` concentrava muitos atributos relacionados. Agrupá-los em subconfigurações deixou a estrutura mais organizada e reduziu o número de atributos, atendendo ao limite do Pylint sem alterar seu funcionamento.

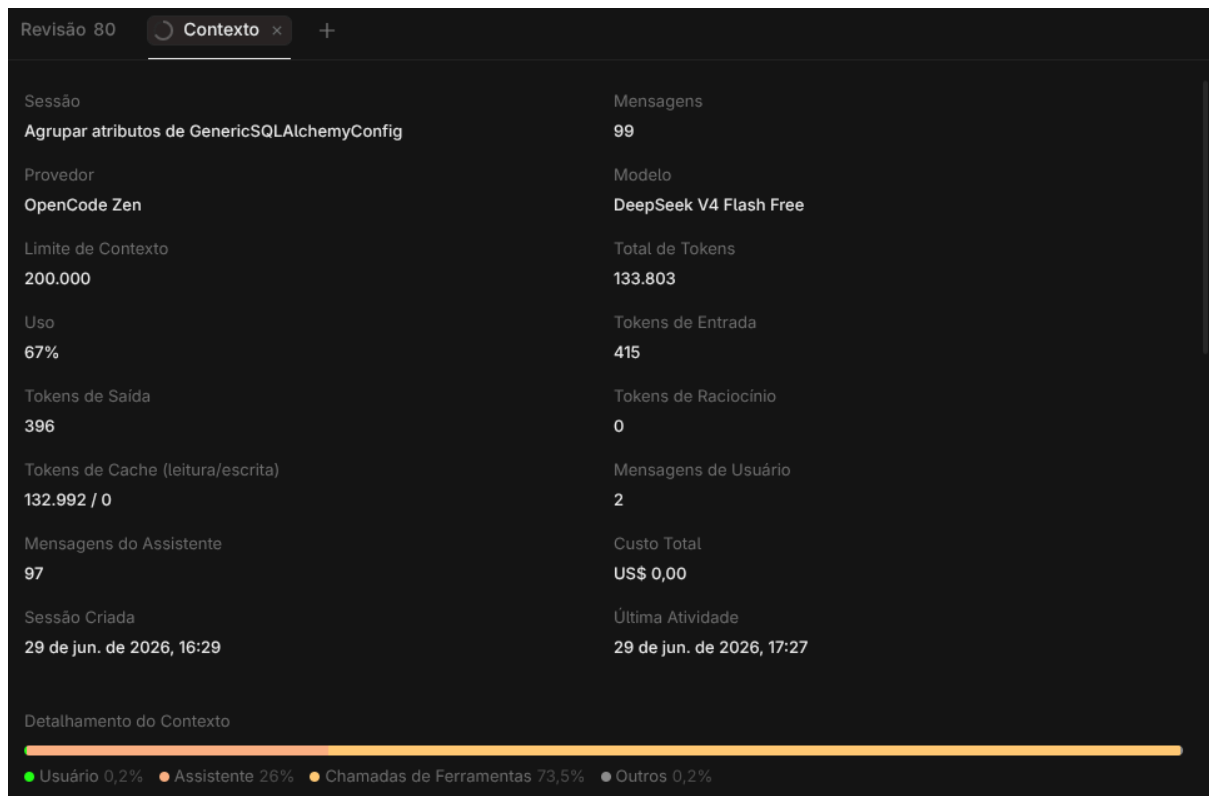
3.2.1 Ocorrência Número 4

Tabela de identificação

module	obj	message
"advanced_alchemy.config.common"	GenericSQLAlchemyConfig",	Too many instance attributes (18/7)",

Link da publicação OpenCode: <https://opncd.ai/share/RhuTHNry>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a classe `GenericSQLAlchemyConfig` concentrava muitas configurações diferentes em um único lugar. Agrupar essas configurações em subconfigurações deixou a classe mais organizada e reduziu o número de atributos, atendendo ao limite do Pylint sem alterar seu funcionamento.

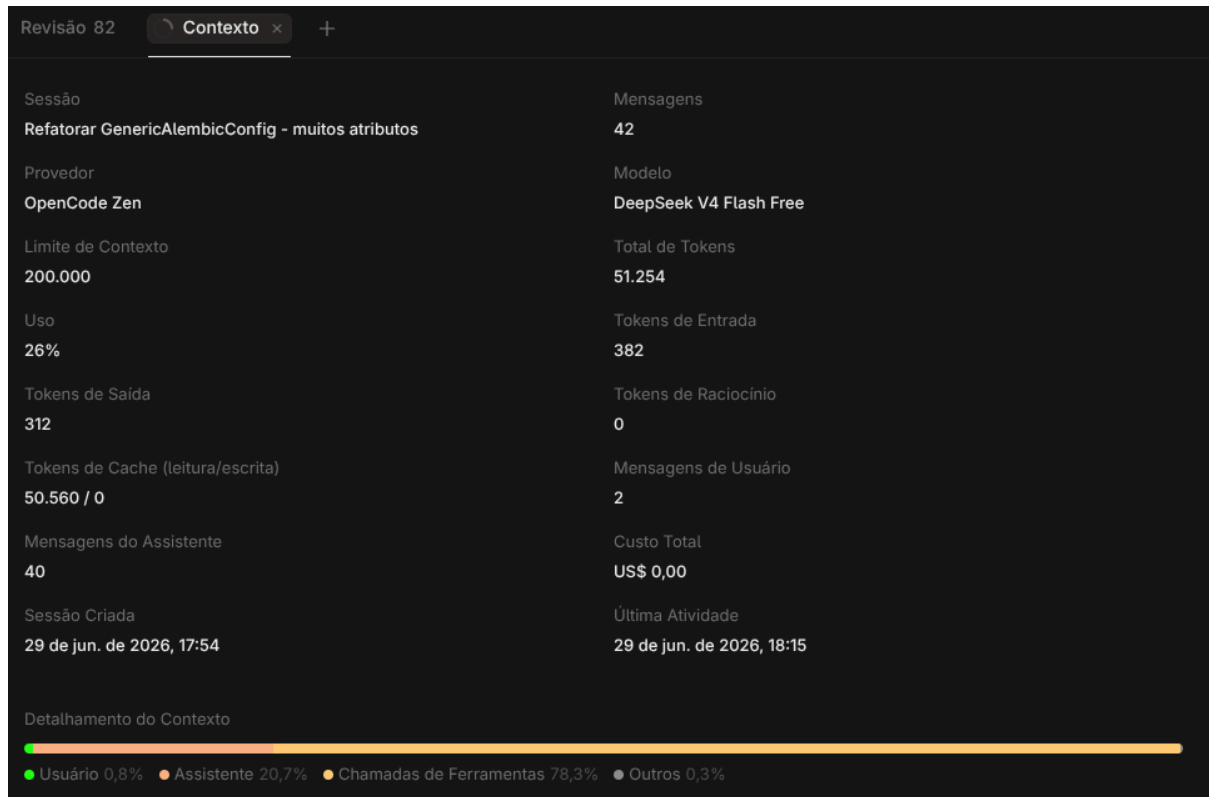
3.2.1 Ocorrência Número 5

Tabela de identificação

module	obj	message
"advanced_alchemy.config.common",	"GenericAlembicConfig", "	"Too many instance attributes (9/7)",

Link da publicação OpenCode: <https://opncd.ai/share/5OVI8nVV>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a classe `GenericAlembicConfig` reunia várias configurações relacionadas em um único lugar. Agrupá-las em subconfigurações deixou a classe mais organizada e reduziu o número de atributos, atendendo ao limite do Pylint sem mudar seu comportamento.

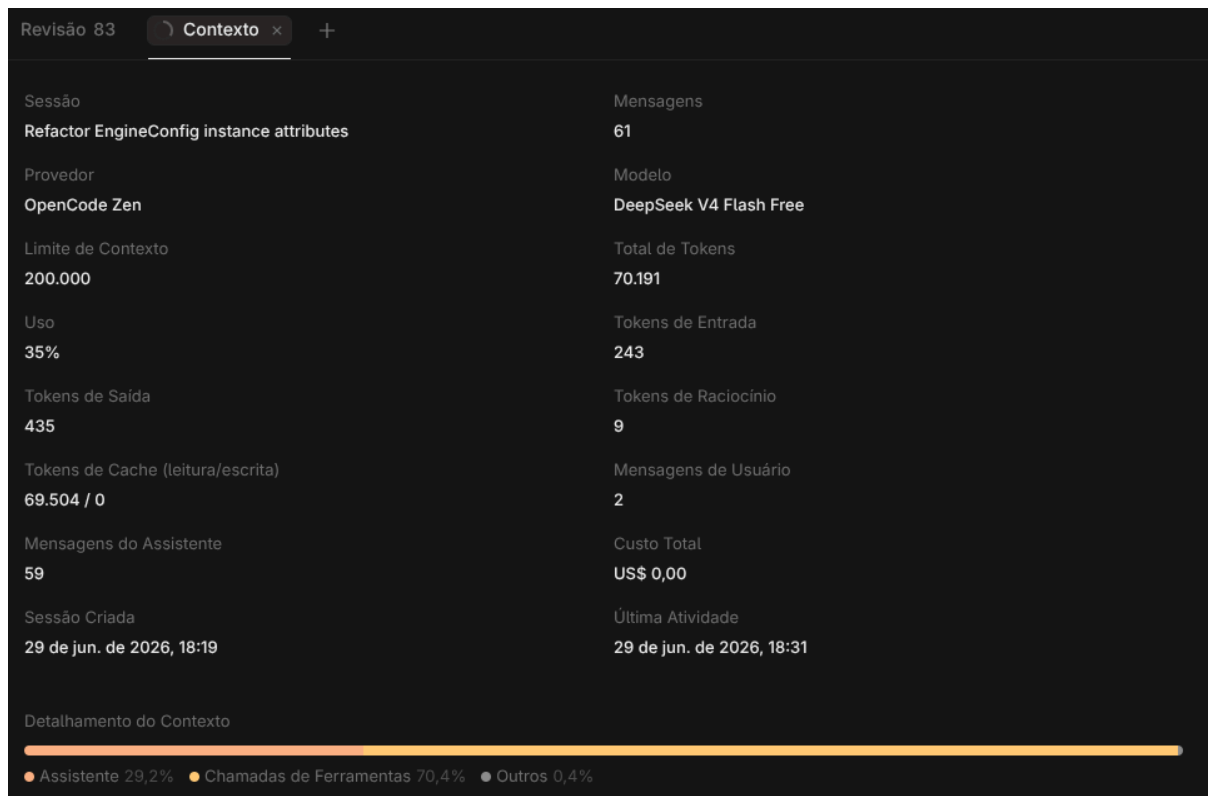
3.2.1 Ocorrência Número 6

Tabela de identificação

module	obj	message
advanced_alchemy.config.engine",	"EngineConfig",	"Too many instance attributes (28/7)",

Link da publicação OpenCode: <https://opncd.ai/share/B8RoCQFI>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque a classe `GenericAlembicConfig` reunia muitas configurações em um único lugar. Ao agrupá-las em subconfigurações, o código ficou mais organizado, reduziu o número de atributos e passou a atender ao limite do Pylint sem alterar seu comportamento.

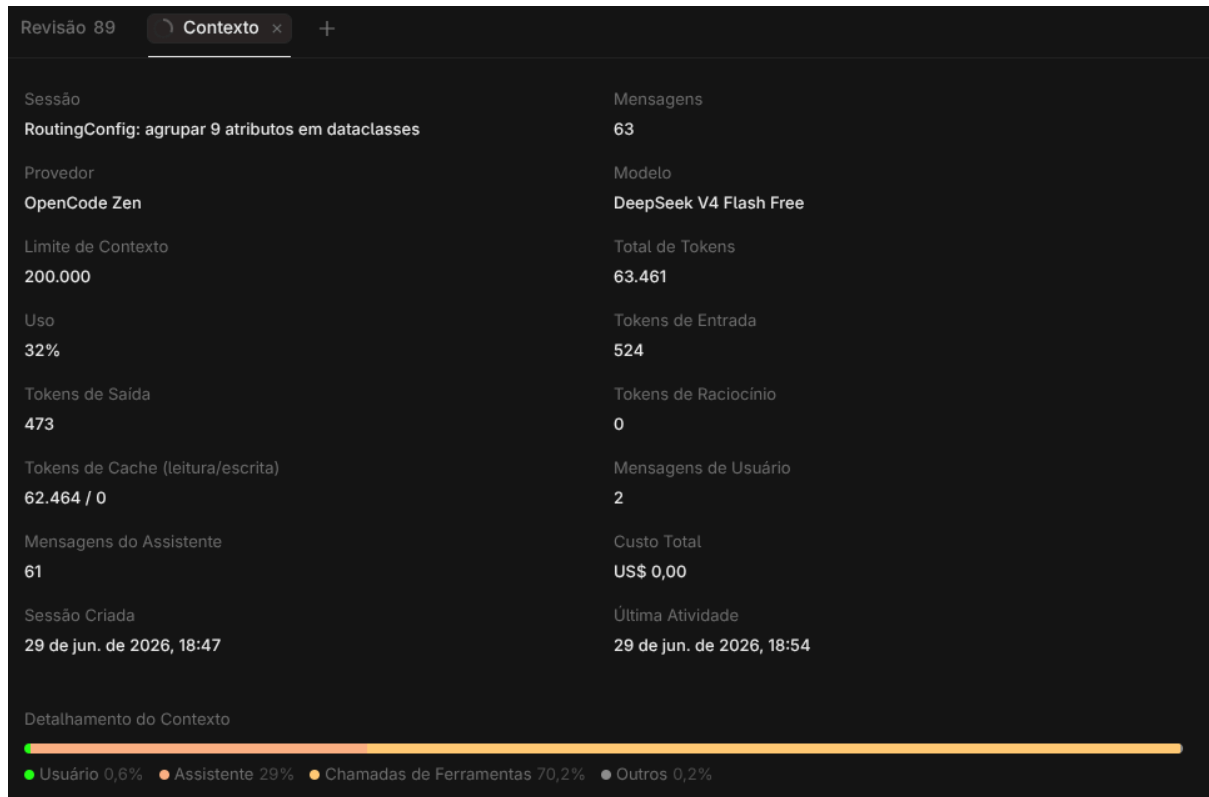
3.2.1 Ocorrência Número 7

Tabela de identificação

module	obj	message
"advanced_alchemy.config.routing",	"RoutingConfig",	Too many instance attributes (9/7)",

Link da publicação OpenCode: <https://opncd.ai/share/NP0eoBO6>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque apenas reorganizou o código, reduzindo a complexidade da classe sem alterar seu comportamento. O ganho em organização e conformidade com o Pylint compensou a mudança.

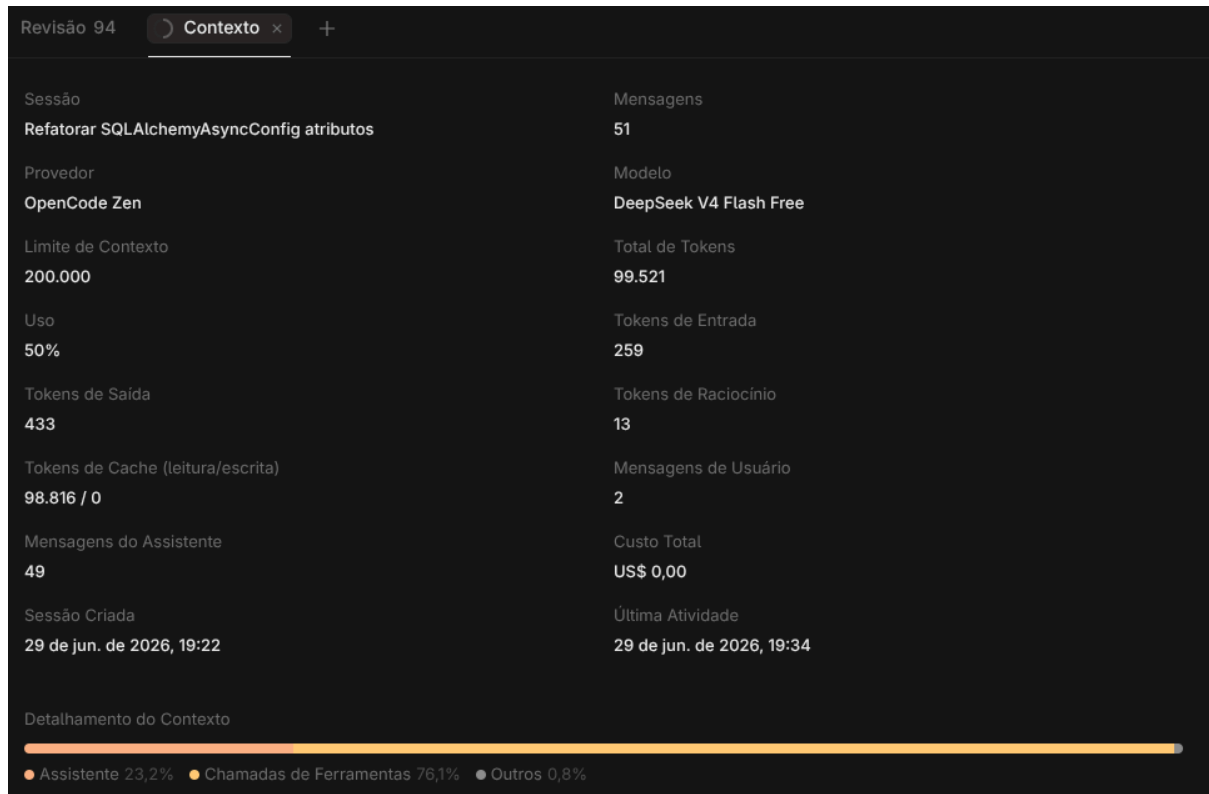
3.2.1 Ocorrência Número 8

Tabela de identificação

module	obj	message
"advanced_alchemy.extensions.litestar.plugins.init.config.asyncio",	"SQLAlchemyAsyncConfig",	"Too many instance attributes (8/7)",

Link da publicação OpenCode: <https://opncd.ai/share/5elvExgy>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque agrupou configurações relacionadas em um único lugar, deixando a classe mais organizada e abaixo do limite do Pylint, sem alterar seu comportamento.

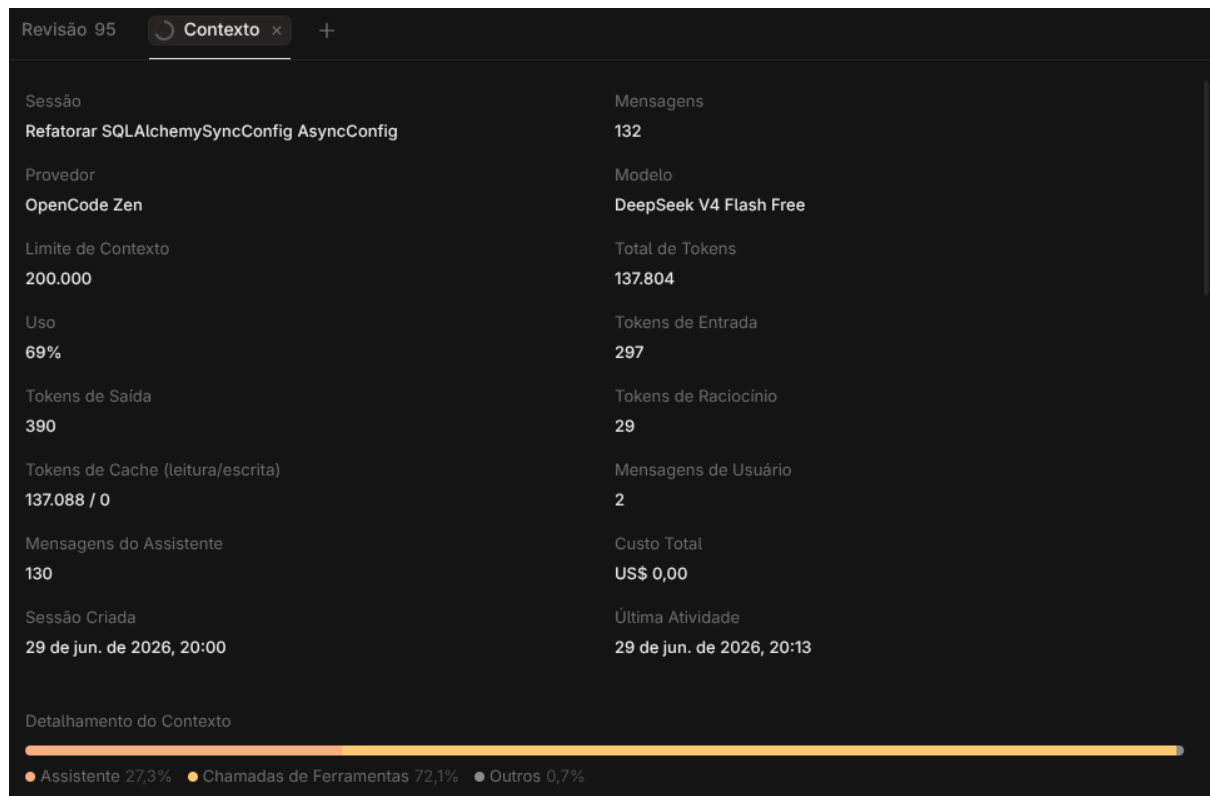
3.2.1 Ocorrência Número 9

Tabela de identificação

module	obj	message
"advanced_alchemy.extensions .littestar.plugins.init.config.sync ",	"SQLAlchemySyncConfig"	"Too many instance attributes (8/7)",

Link da publicação OpenCode: <https://opncd.ai/share/PxBp2gs1>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque agrupou as chaves de sessão em uma configuração específica, deixando a classe mais organizada e reduzindo o número de atributos, sem alterar seu comportamento.

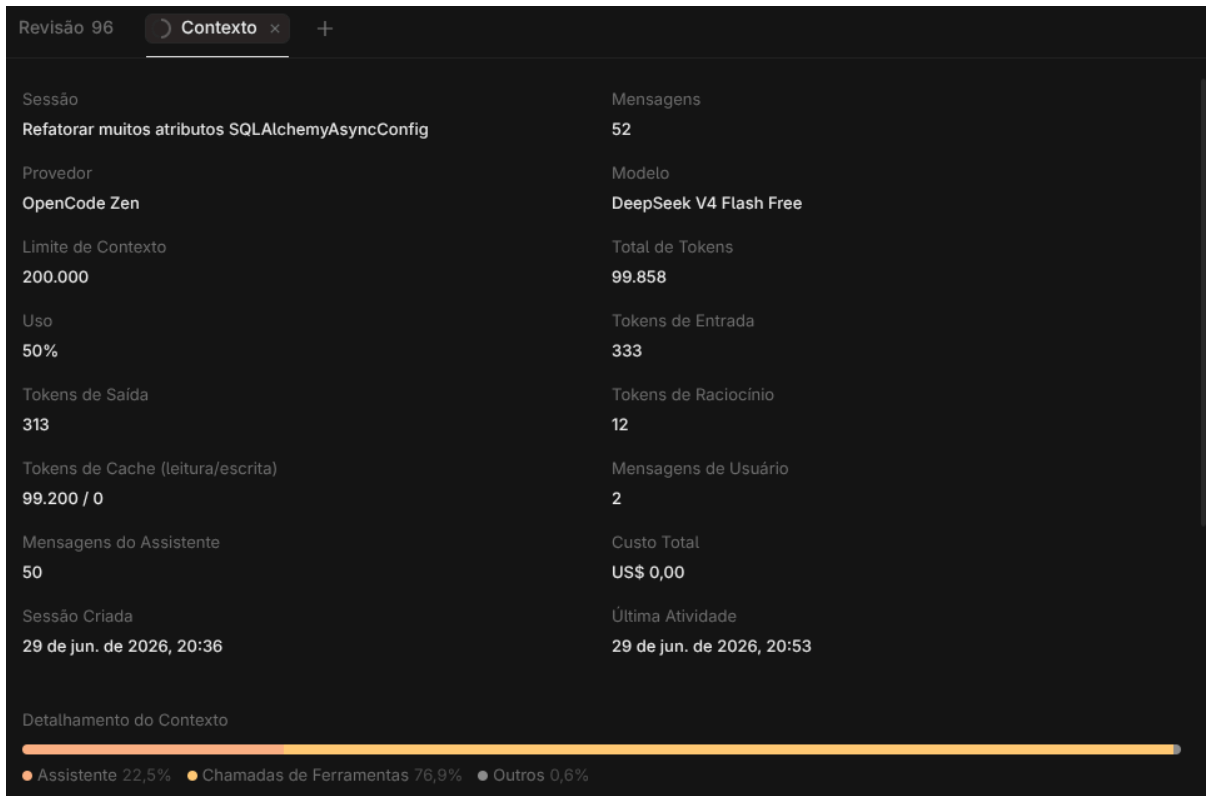
3.2.1 Ocorrência Número 10

Tabela de identificação

module	obj	message
"advanced_alchemy.extensions .sanic.config",	"SQLAlchemyAsyncConfig",	"Too many instance attributes (8/7)",

Link da publicação OpenCode: <https://opncd.ai/share/FziX2DL7>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque agrupou as chaves de estado da aplicação em uma única configuração, deixando a classe mais organizada e reduzindo o número de atributos, sem alterar seu comportamento.

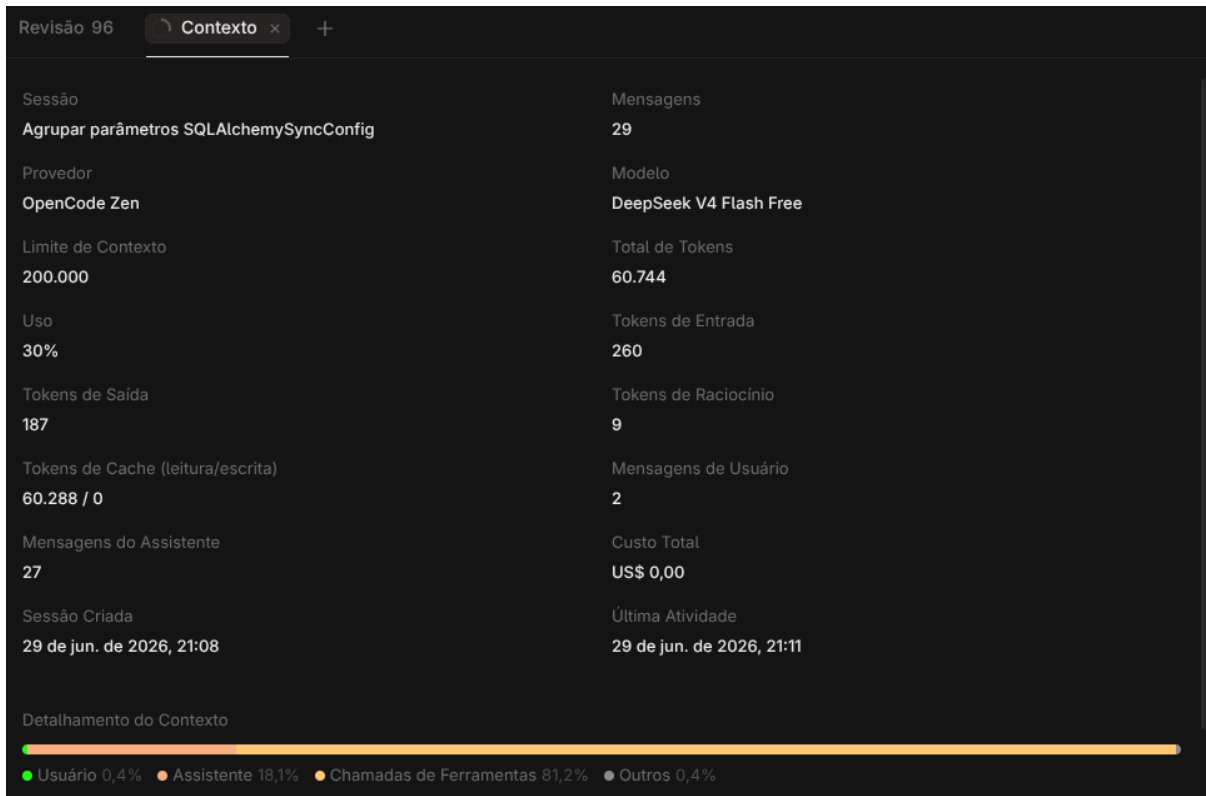
3.2.1 Ocorrência Número 11

Tabela de identificação

module	obj	message
advanced_alchemy.extensions. sanic.config",	,"SQLAlchemySyncConfig",	"Too many instance attributes (8/7)",

Link da publicação OpenCode: <https://opncd.ai/share/ay5pfW31>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque seguiu o mesmo padrão já usado na versão assíncrona, agrupando as chaves em uma única configuração. Isso deixou a classe mais organizada e resolveu o alerta do Pylint sem mudar seu comportamento.

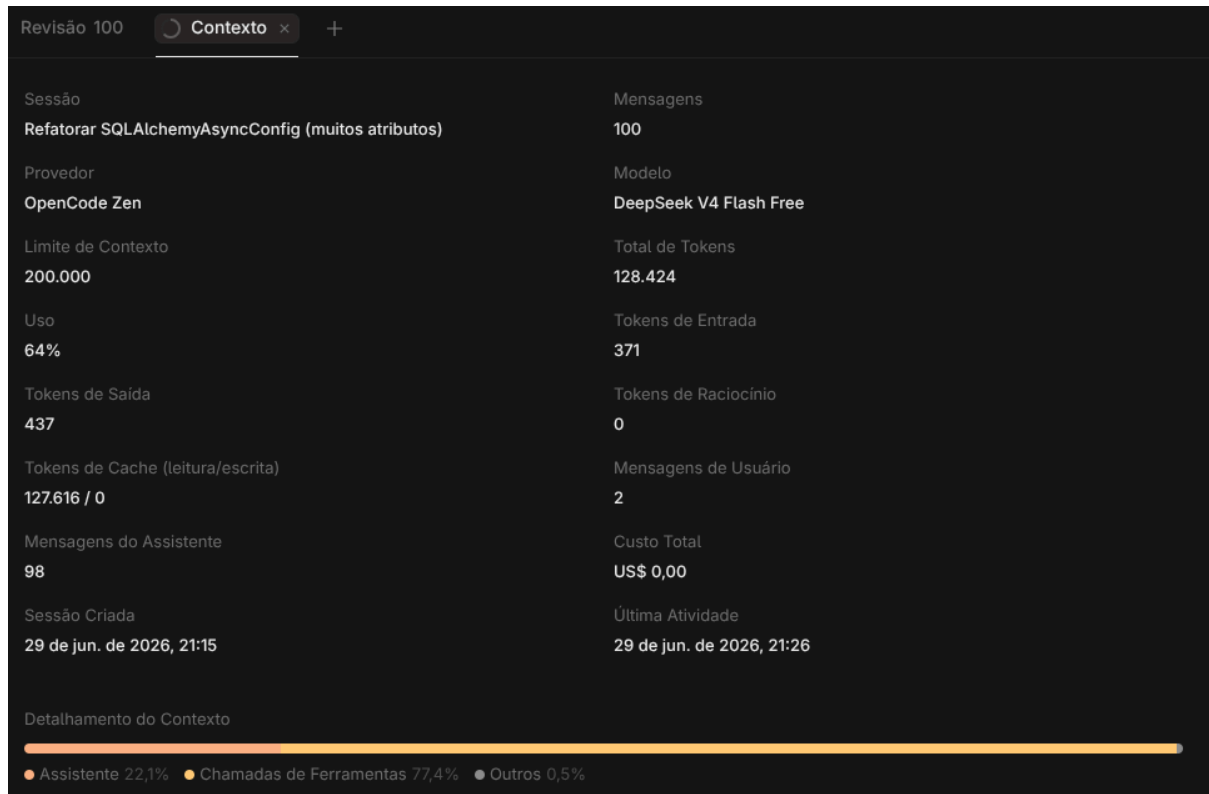
3.2.1 Ocorrência Número 12

Tabela de identificação

module	obj	message
"advanced_alchemy.extensions.starlette.config",	"SQLAlchemyAsyncConfig",	"Too many instance attributes (8/7)",

Link da publicação OpenCode: <https://opncd.ai/share/fRGjBqwo>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque seguiu um padrão já usado em outra extensão do projeto, agrupando as chaves de estado em uma única configuração. Isso deixou a classe mais organizada e resolveu o alerta do Pylint sem alterar seu comportamento.

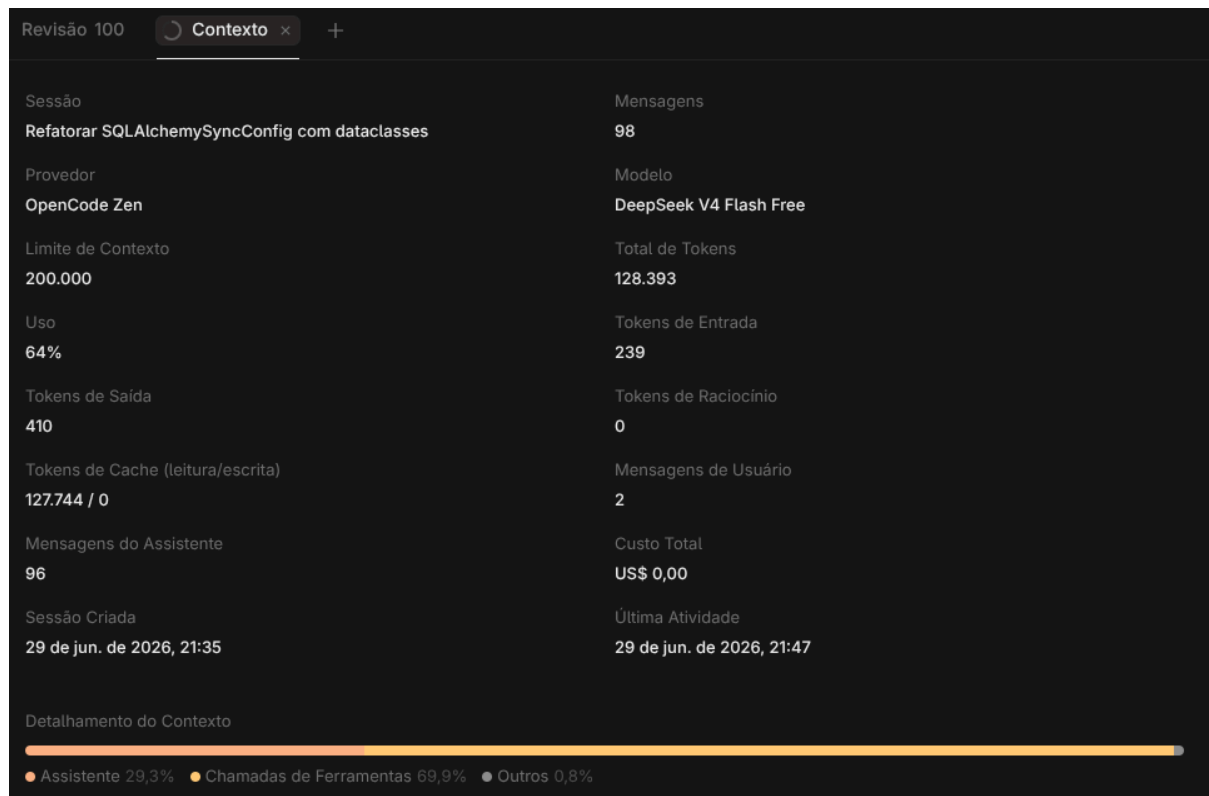
3.2.1 Ocorrência Número 13

Tabela de identificação

module	obj	message
"advanced_alchemy.extensions.starlette.config",	"SQLAlchemySyncConfig",	"Too many instance attributes (8/7)",

Link da publicação OpenCode: <https://opncd.ai/share/pLP4wSpD>

Print consumo de token:



Análise da refatoração: A refatoração valeu a pena porque agrupou as configurações específicas do Starlette em uma única estrutura. Isso deixou as classes mais organizadas e resolveu o alerta do Pylint sem alterar seu comportamento.

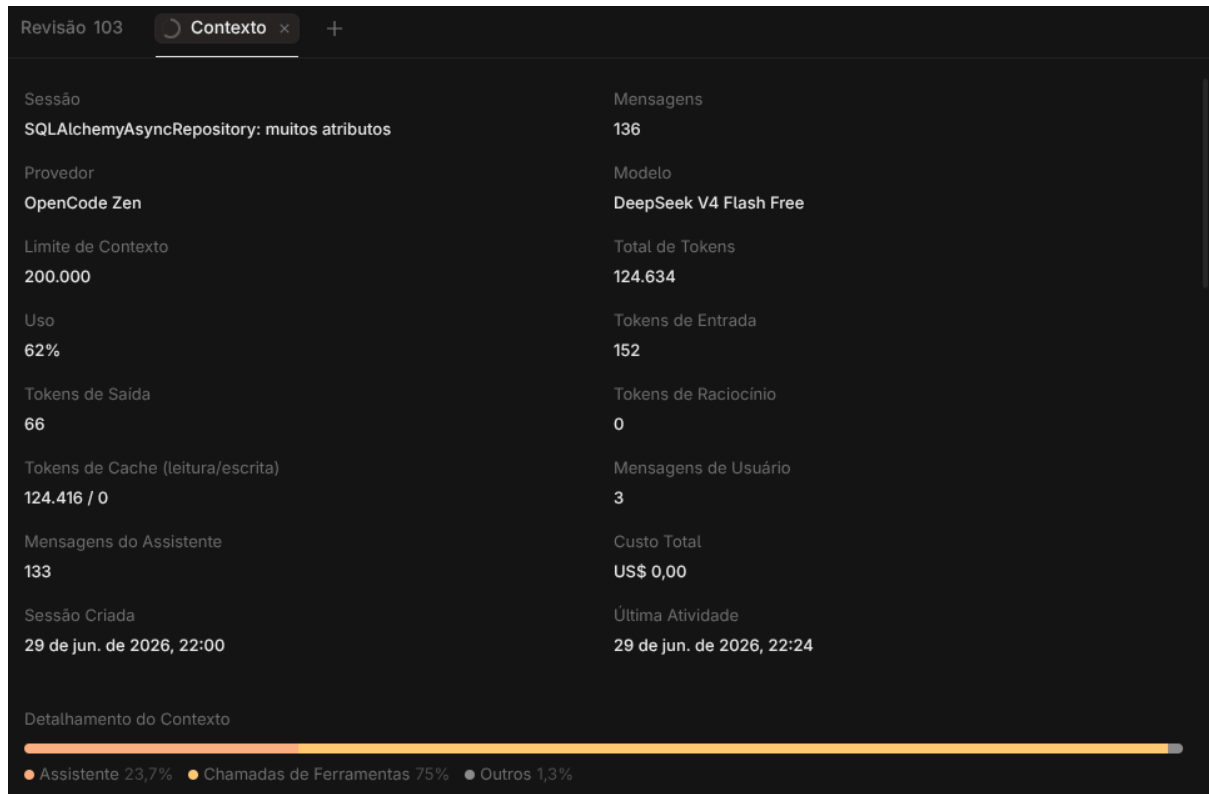
3.2.1 Ocorrência Número 14

Tabela de identificação

module	obj	message
"advanced_alchemy.repository._async",	"SQLAlchemyAsyncRepository",	"Too many instance attributes (19/7)",

Link da publicação OpenCode: <https://opncd.ai/share/tsTPCOz5>

Print consumo de token:



Análise da refatoração: A refatoração ficou boa e resolveu o problema de forma organizada. Pode prosseguir com essas alterações.

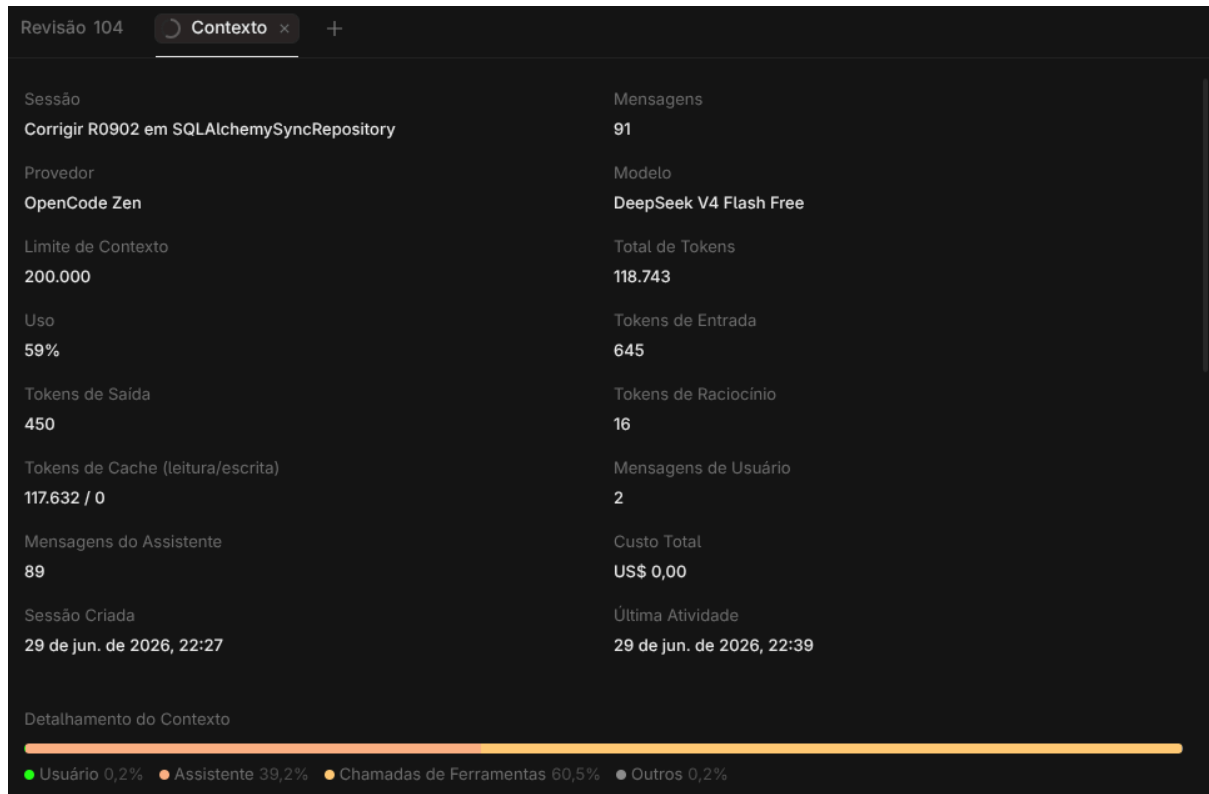
3.2.1 Ocorrência Número 15

Tabela de identificação

module	obj	message
"advanced_alchemy.repository._sync",	"SQLAlchemySyncRepository",	"Too many instance attributes (19/7)",

Link da publicação OpenCode: <https://opncd.ai/share/gM4BPTI7>

Print consumo de token:



Análise da refatoração: A abordagem está consistente com a refatoração anterior e mantém o código mais organizado. Pode prosseguir com a implementação.

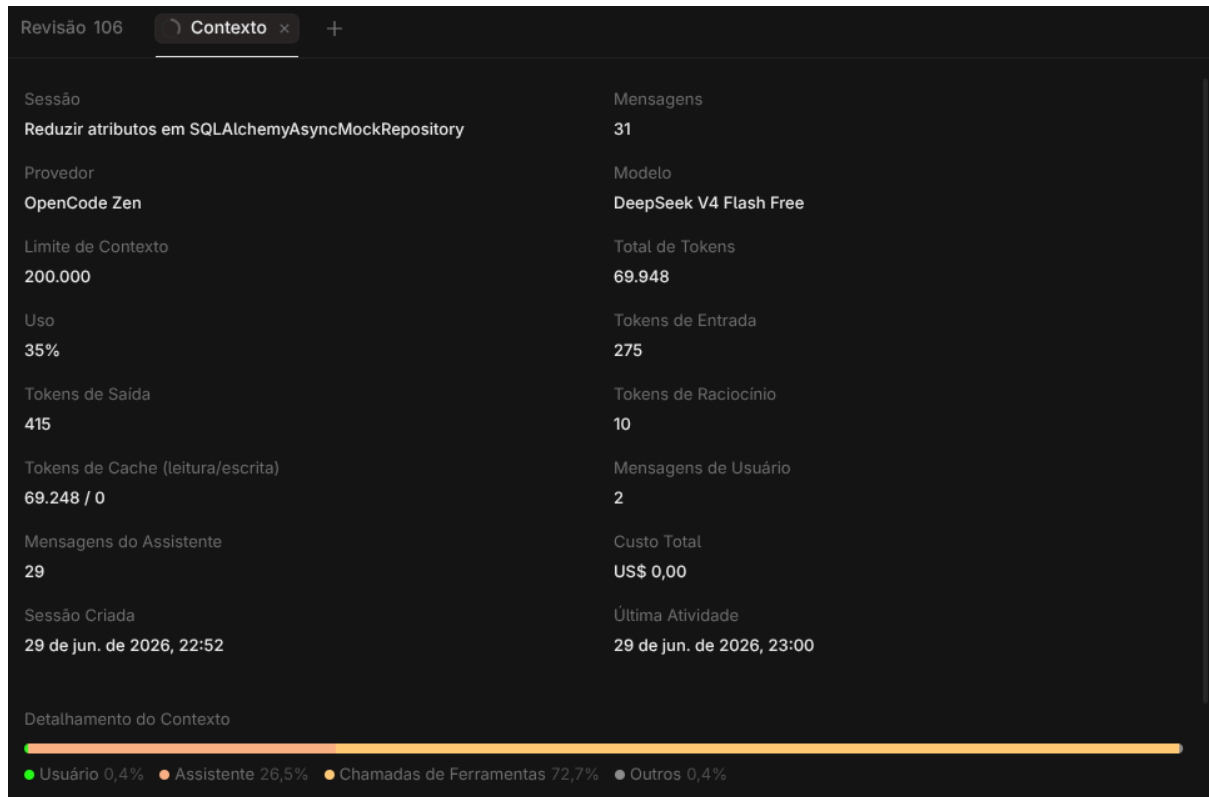
3.2.1 Ocorrência Número 16

Tabela de identificação

module	obj	message
"advanced_alchemy.repository.memory._async",	"SQLAlchemyAsyncMockRepository"	"Too many instance attributes (15/7)",

Link da publicação OpenCode: <https://opncl.ai/share/CNX4GPjJ>

Print consumo de token:



Análise da refatoração: A abordagem segue o mesmo padrão adotado nos repositórios anteriores e mantém a compatibilidade. Pode prosseguir com a implementação.

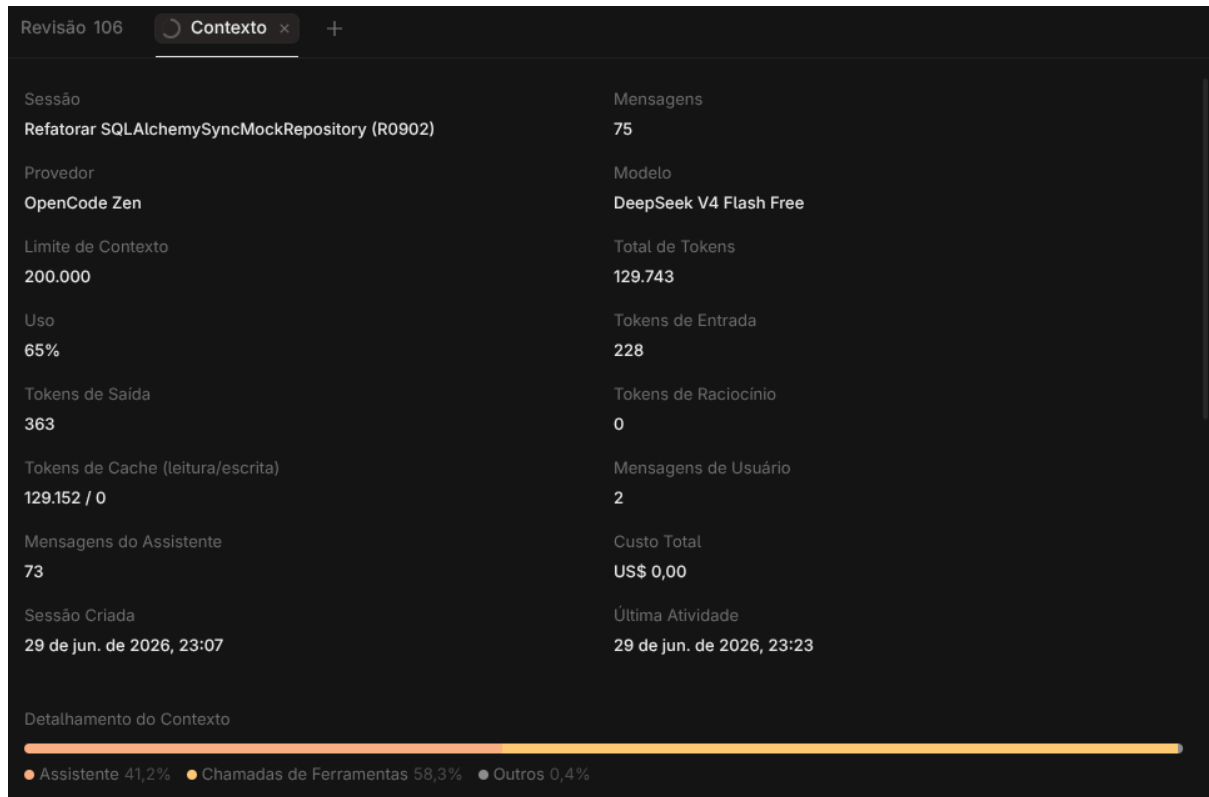
3.2.1 Ocorrência Número 17

Tabela de identificação

module	obj	message
"advanced_alchemy.repository.memory._sync",	"SQLAlchemySyncMockRepository",	"Too many instance attributes (15/7)",

Link da publicação OpenCode: <https://opncd.ai/share/w0KEXIeY>

Print consumo de token:



Análise da refatoração: A abordagem segue a mesma estrutura aplicada na versão assíncrona, mantendo consistência e reduzindo a quantidade de atributos da classe. Pode prosseguir com essa refatoração.

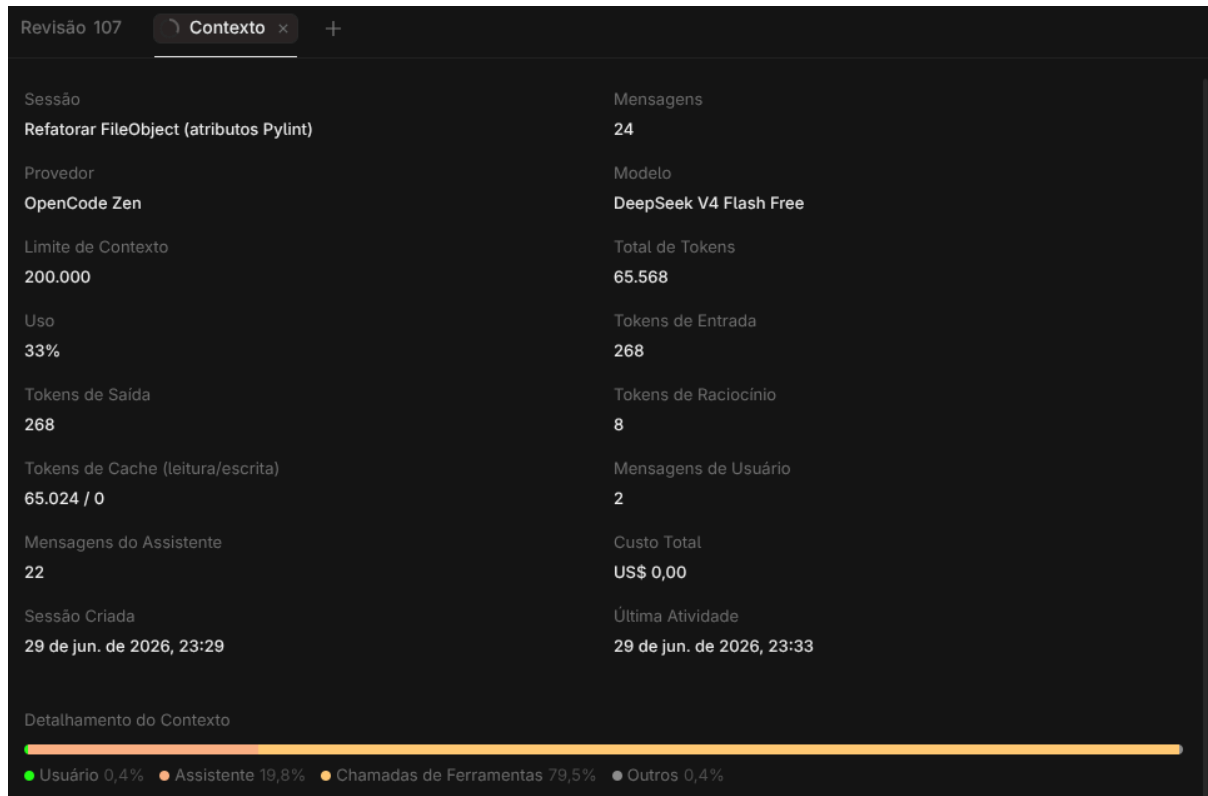
3.2.1 Ocorrência Número 18

Tabela de identificação

module	obj	message
"advanced_alchemy.types.file_object.file",	"FileObject",	"Too many instance attributes (13/7)",

Link da publicação OpenCode: <https://opncd.ai/share/qKAONrwM>

Print consumo de token:



Análise da refatoração: A proposta está consistente e preserva a compatibilidade da API. Pode prosseguir com a implementação conforme o plano.

3.3 too-many-branches

Primeiramente, informe a quantidade de ocorrências desse tipo de smell.

Quantidade de ocorrências: 5

3.3.1 Ocorrência Número 1

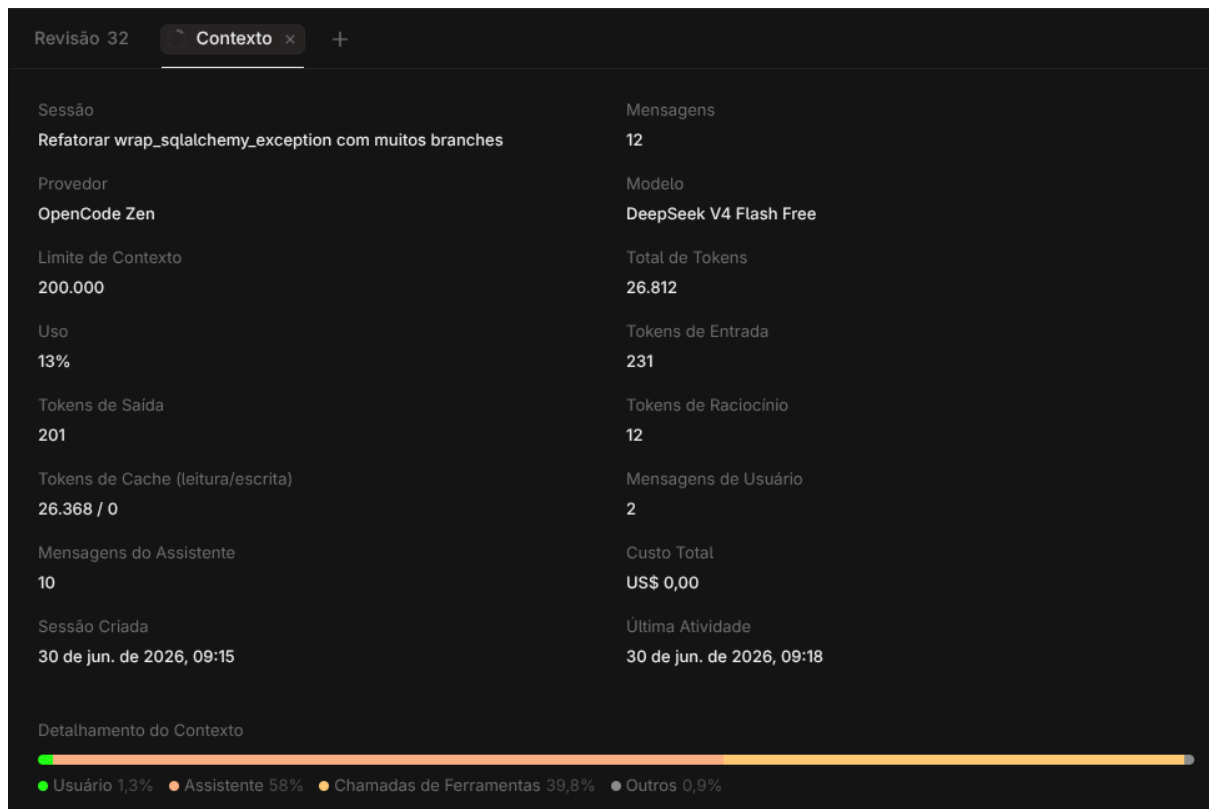
Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

module	obj	message
"advanced_alchemy.exceptions",	"wrap_sqlalchemy_exception",	"Too many branches (26/12)",

Link da publicação OpenCode: <https://opncd.ai/share/AMEKyNJ0>

Print consumo de token:



Análise da refatoração: A abordagem deixa o tratamento de exceções mais organizado e facilita a manutenção. Pode prosseguir com a implementação conforme o plano.

3.3.1 Ocorrência Número 2

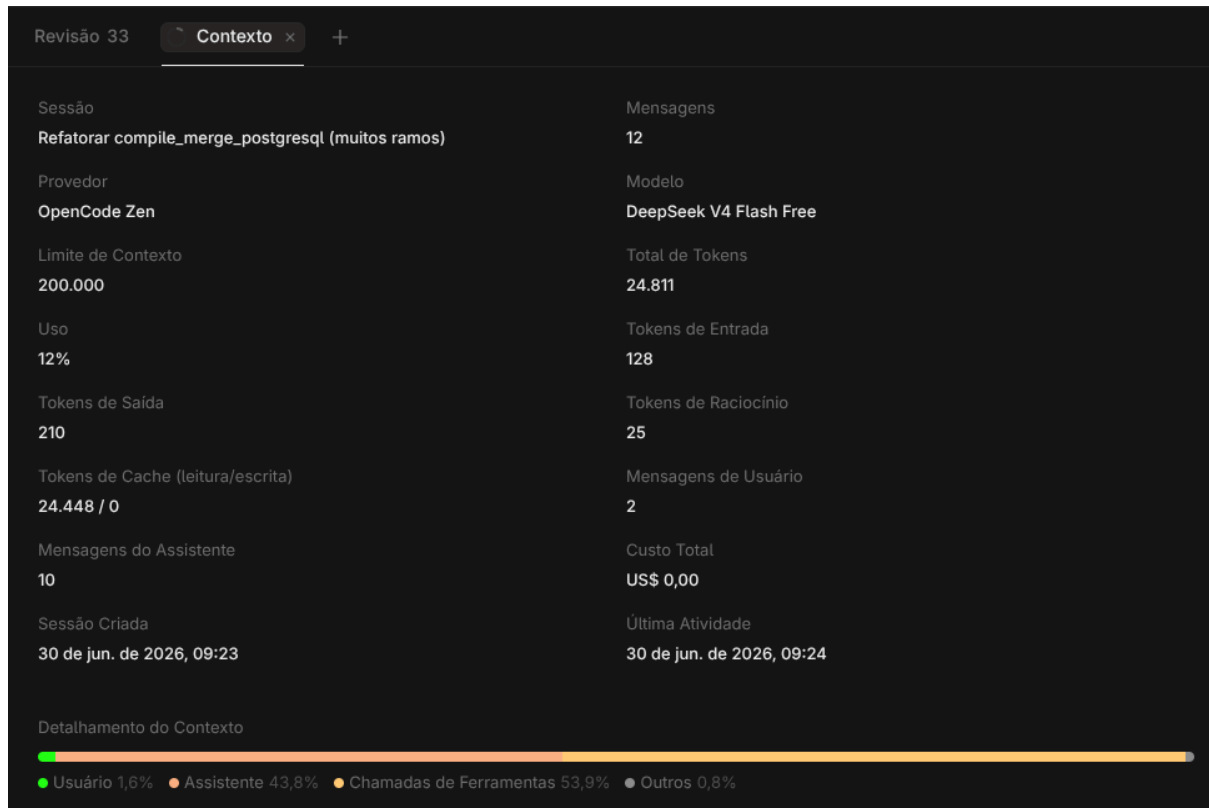
Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

module	obj	message
"advanced_alchemy.operations",	"compile_merge_postgresql",	"Too many branches (13/12)",

Link da publicação OpenCode: <https://opncd.ai/share/3oP4AZS2>

Print consumo de token:



Análise da refatoração: A proposta simplifica a função sem alterar o comportamento e melhora a legibilidade do código. Pode prosseguir com a implementação.

3.3.1 Ocorrência Número 2

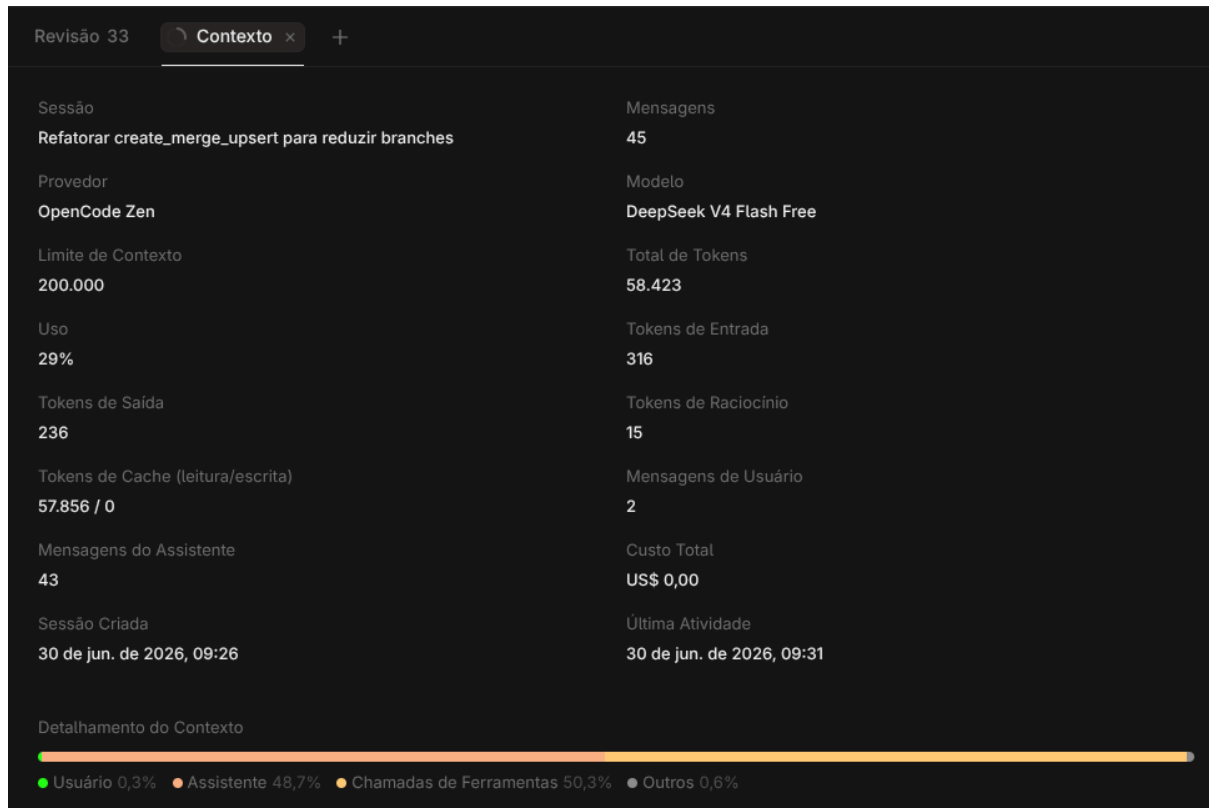
Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

module	obj	message
advanced_alchemy.operations"	"OnConflictUpsert.create_merge_upsert",	"Too many branches (23/12)",
,		

Link da publicação OpenCode: <https://opncd.ai/share/iKaEW1jP>

Print consumo de token:



Análise da refatoração: A proposta deixa o método mais organizado e mantém o padrão já adotado nas refatorações anteriores. Pode prosseguir com a implementação.

3.3.1 Ocorrência Número 3

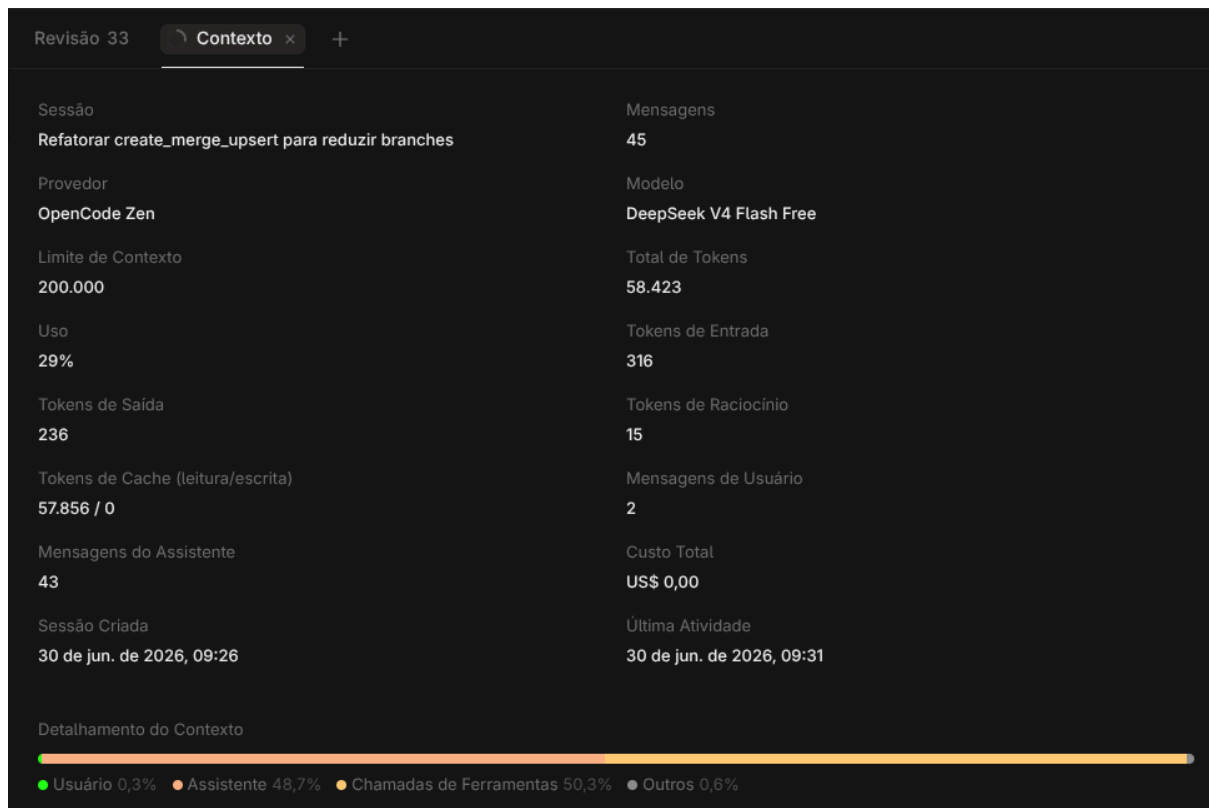
Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

module	obj	message
"advanced_alchemy.extensions .littestar.dto",	SQLAlchemyDTO.generate_field_definitions",	"Too many branches (16/12)",

Link da publicação OpenCode: <https://opncd.ai/share/iKaEW1jP>

Print consumo de token:



Análise da refatoração: A proposta deixa o método mais organizado e mantém o padrão já adotado nas refatorações anteriores. Pode prosseguir com a implementação.

3.3.1 Ocorrência Número 4

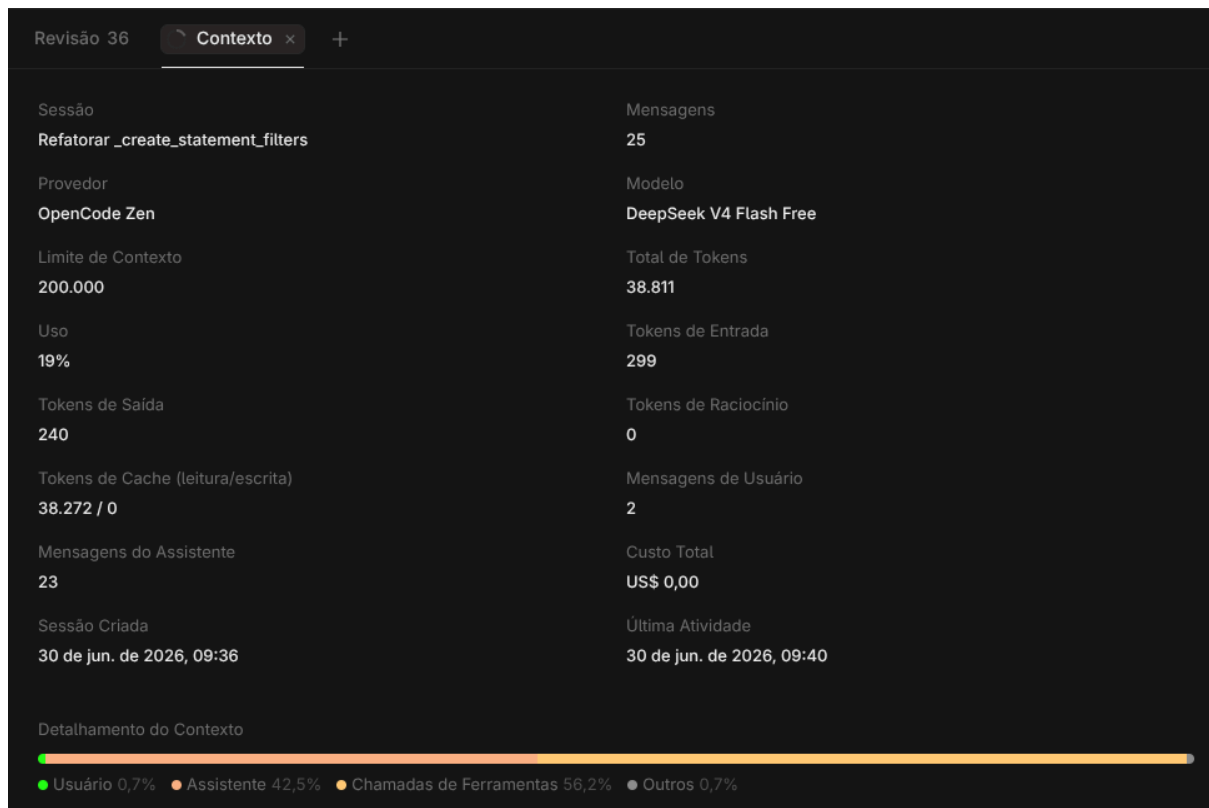
Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

module	obj	message
"advanced_alchemy.extensions .littestar.providers",	_create_statement_filters",	"Too many branches (15/12)",

Link da publicação OpenCode: <https://opncd.ai/share/If7sSrCS>

Print consumo de token:



Análise da refatoração: A proposta está consistente e reduz a complexidade da função sem alterar o comportamento. Pode prosseguir com a implementação conforme o plano.

3.3.1 Ocorrência Número 5

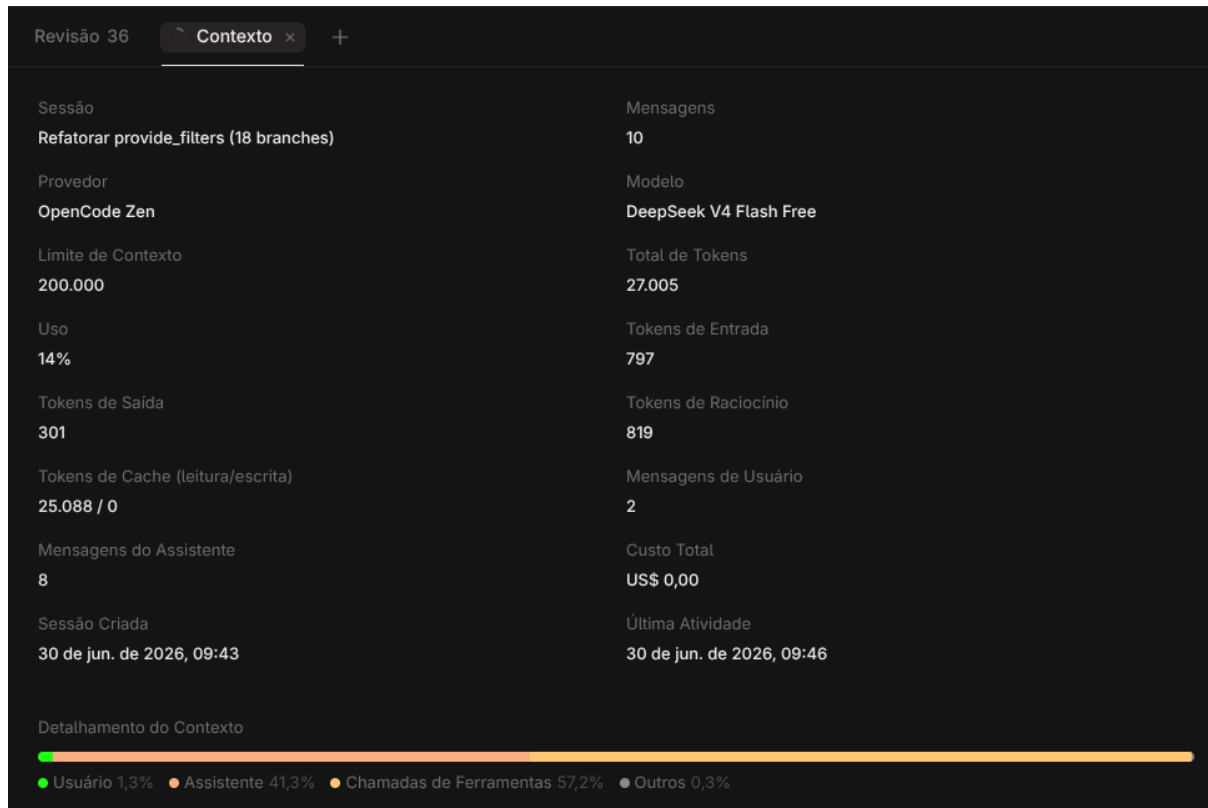
Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

module	obj	message
"advanced_alchemy.extensions.litestar.providers",	_create_filter_aggregate_function.provide_filters",	"Too many branches (18/12)",

Link da publicação OpenCode: <https://opncd.ai/share/y2IAI1lv>

Print consumo de token:



Análise da refatoração: A análise faz sentido. Como a refatoração já foi aplicada e a função está delegando a lógica para helpers, não há mais alterações necessárias.

4. Métricas Após a Refatoração

Para as métricas após a refatoração você deve adicionar os mesmos tipos de informações que foram apresentadas anteriormente. Atenção, certifique-se de que cada um dos dados estão nas pastas metrics-after-...

4.1 Radon Após a Refatoração

4.1.1 Complexidade Ciclômática por Arquivo Após a Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
advanced_alchemy/types/password_hash/base.py	15	1.4	3	21	A	verify_and_update
advanced_alchemy/routing/selectors.py	9	1.33	2	12	A	__init__
advanced_alchemy/repository/_typing.py	4	1.25	2	5	A	is_numpy_array_stub
advanced_alchemy/types/mutables.py	17	1.24	3	21	A	coerce
advanced_alchemy/_typing.py	21	1.24	3	26	A	model_dump

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
advanced_alchemy/__main__.py	1	1.0	1	1	A	run_cli
advanced_alchemy/cache/_null.py	19	1.0	1	19	A	get
advanced_alchemy/cache/config.py	1	1.0	1	1	A	_default_arguments
advanced_alchemy/extensions/flask/cli.py	2	1.0	1	2	A	get_database_migration_plugin
advanced_alchemy/extensions/litestar/cli.py	2	1.0	1	2	A	get_database_migration_plugin

4.1.2 Complexidade Ciclômática por Função Após a Refatoração

Apresente as funções do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
advanced_alchemy/_typing.py	method	BaseModelLike.model_dump	A	3
advanced_alchemy/types/mutables.py	method	MutableList.coerce	A	3
advanced_alchemy/types/password_hash/base.py	method	HashedPassword.verify_and_update	A	3
advanced_alchemy/routing/context.py	function	should_use_primary	A	2
advanced_alchemy/repository/_typing.py	function	is_numpy_array_stub	A	2

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
advanced_alchemy/__main__.py	function	run_cli	A	1
advanced_alchemy/_listeners.py	function	set_async_context	A	1
advanced_alchemy/_listeners.py	function	reset_async_context	A	1
advanced_alchemy/_listeners.py	function	is_async_context	A	1
advanced_alchemy/_listeners.py	method	BaseFileObjectListener.is_listener_enabled	A	1

4.1.3 Índice de Manutenibilidade Após a Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	mi	rank_mi
advanced_alchemy/repository/_async.py	0.00	C
advanced_alchemy/repository/_sync.py	0.00	C
advanced_alchemy/repository/memory/_async.py	20.18	A
advanced_alchemy/repository/memory/_sync.py	20.35	A
advanced_alchemy/utils/serialization.py	21.03	A

Tabela de melhores métricas

arquivo	mi	rank_mi
advanced_alchemy/types/encrypted_string.py	44.61	A
advanced_alchemy/extensions/sanic/config.py	44.17	A
advanced_alchemy/config/common.py	43.82	A
advanced_alchemy/operations.py	43.14	A
advanced_alchemy/extensions/litestar/dto.py	42.38	A

4.1.4 Métricas Brutas Após a Refatoração

Essas métricas podem ser encontradas no arquivo *raw_por_arquivo_e_total_depois.csv* Traga apenas o total, que se encontra no final do csv.

loc total	lloc total	sloc total	comments total	multi
40.197	15.955	23.867	1.559	6.430

4.2 CodeCarbon Após a Refatoração

Preencha a tabela abaixo com as métricas ambientais depois da refatoração.

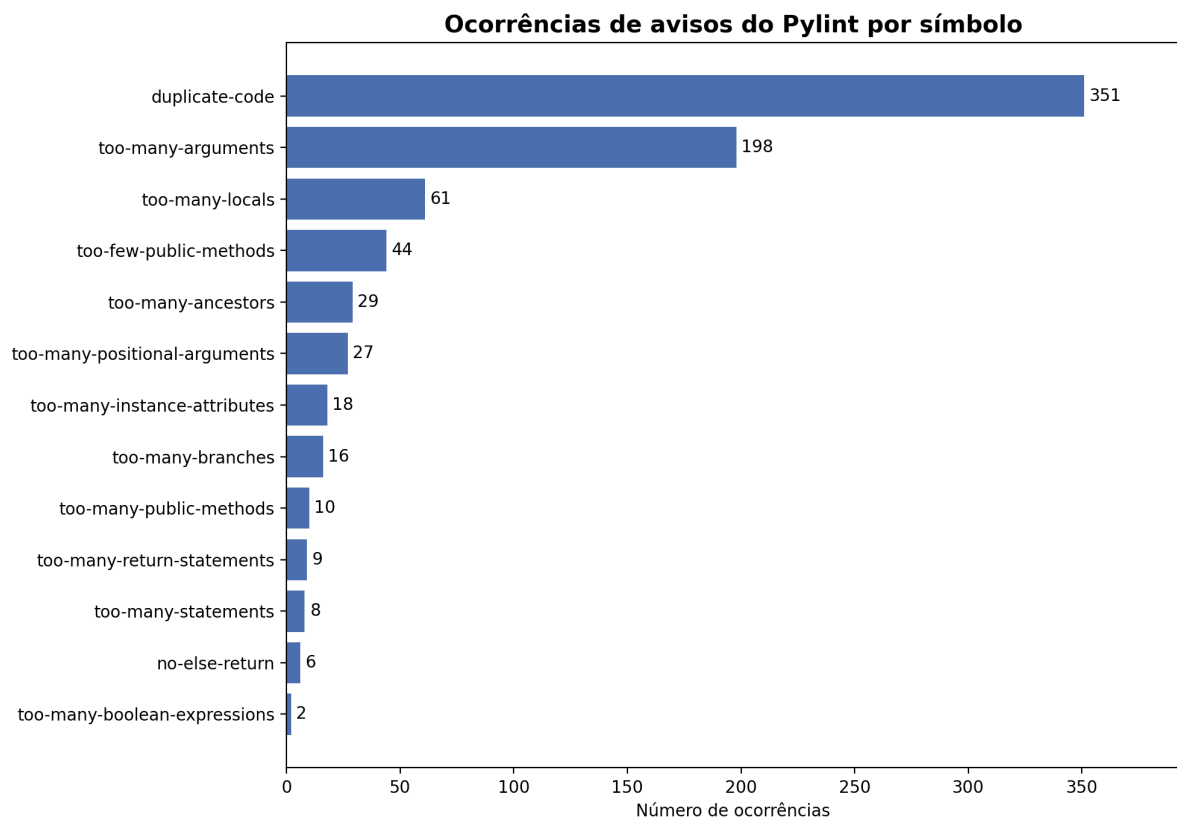
duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
2.621043	0.000001400267	0.000000534240	0.000002785790	0.000011452088	0.000014237878

4.3 Pylint Após a Refatoração

4.3.1 Distribuição do Smells Após a Refatoração

A partir do JSON *pylint_ranking_smells_depois.json* gere um gráfico para representar a distribuição dos code smells depois da refatoração. O gráfico precisa ter um bom contraste para facilitar a

visualização.

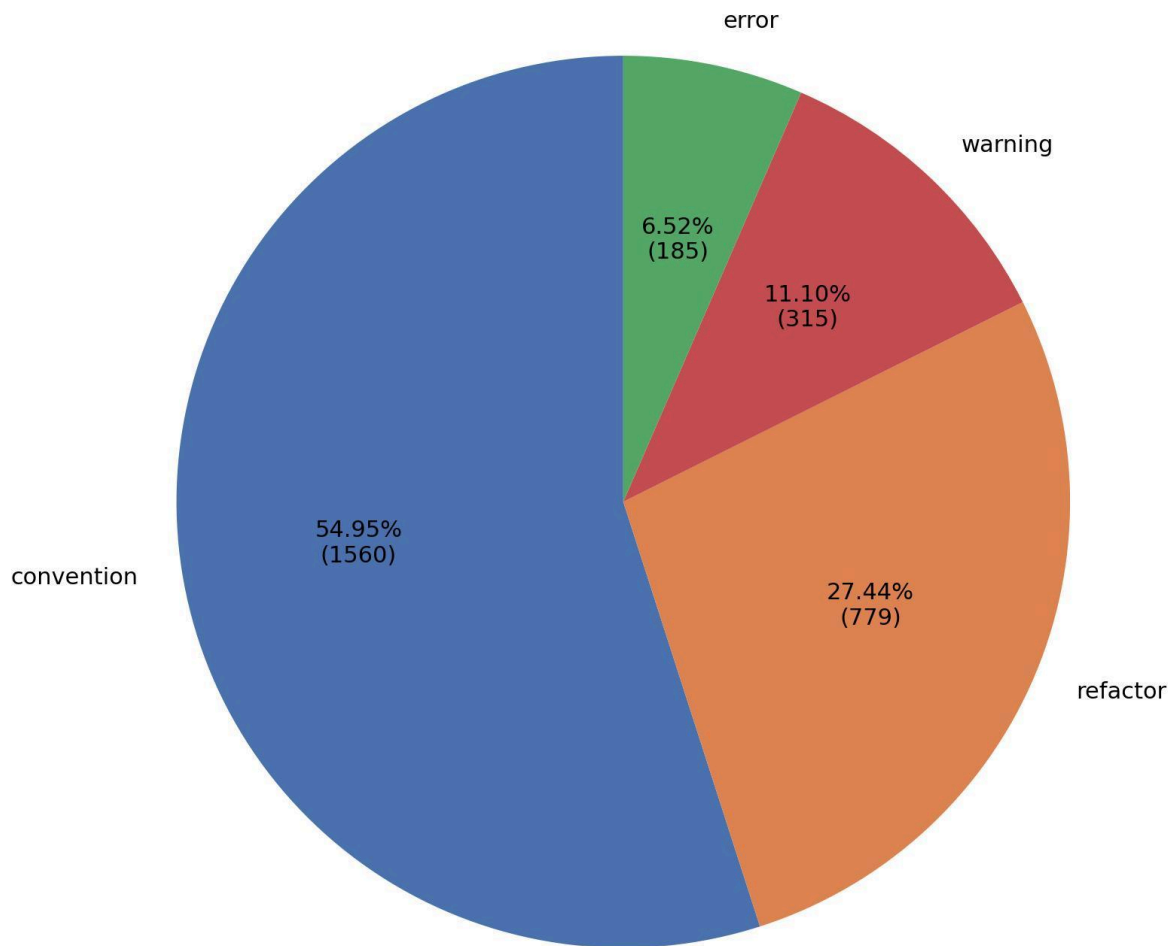


4.3.2 Distribuição dos Problemas de Código Após a Refatoração

A partir do JSON `pylint_distribuicao_categorias_depois.json` gere um gráfico para representar a distribuição entre as 4 categorias de problemas de código depois da refatoração. O gráfico precisa ter

um bom contraste para facilitar a visualização.

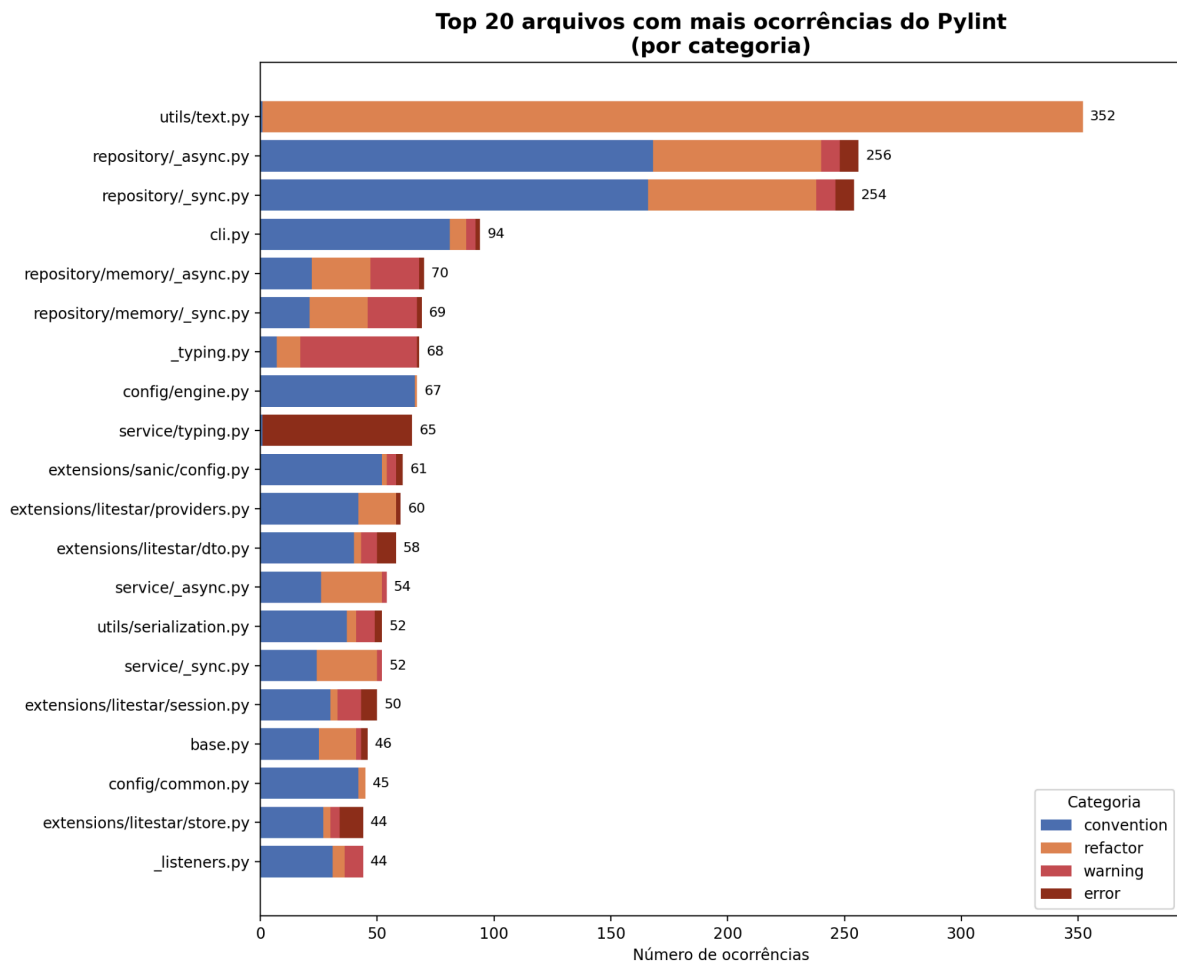
Distribuição de ocorrências por categoria (Pylint)



4.3.3 Arquivos mais Críticos Após a Refatoração

A partir do JSON `pylint_arquivos_criticos_depois.json` gere um gráfico para representar a distribuição dos 10 arquivos mais críticos depois da refatoração. O gráfico precisa ter um bom contraste para

facilitar a visualização.



4.3.4 Score Projeto Após a Refatoração

A partir do text `pylint_score_depois.txt` informe o score que o pylint deu ao projeto depois da refatoração. Não precisa ser uma imagem, fica a critério de vocês.

Score Pós-Refatoração: 7.38/10

Score Pré-Refatoração: 7.09/10

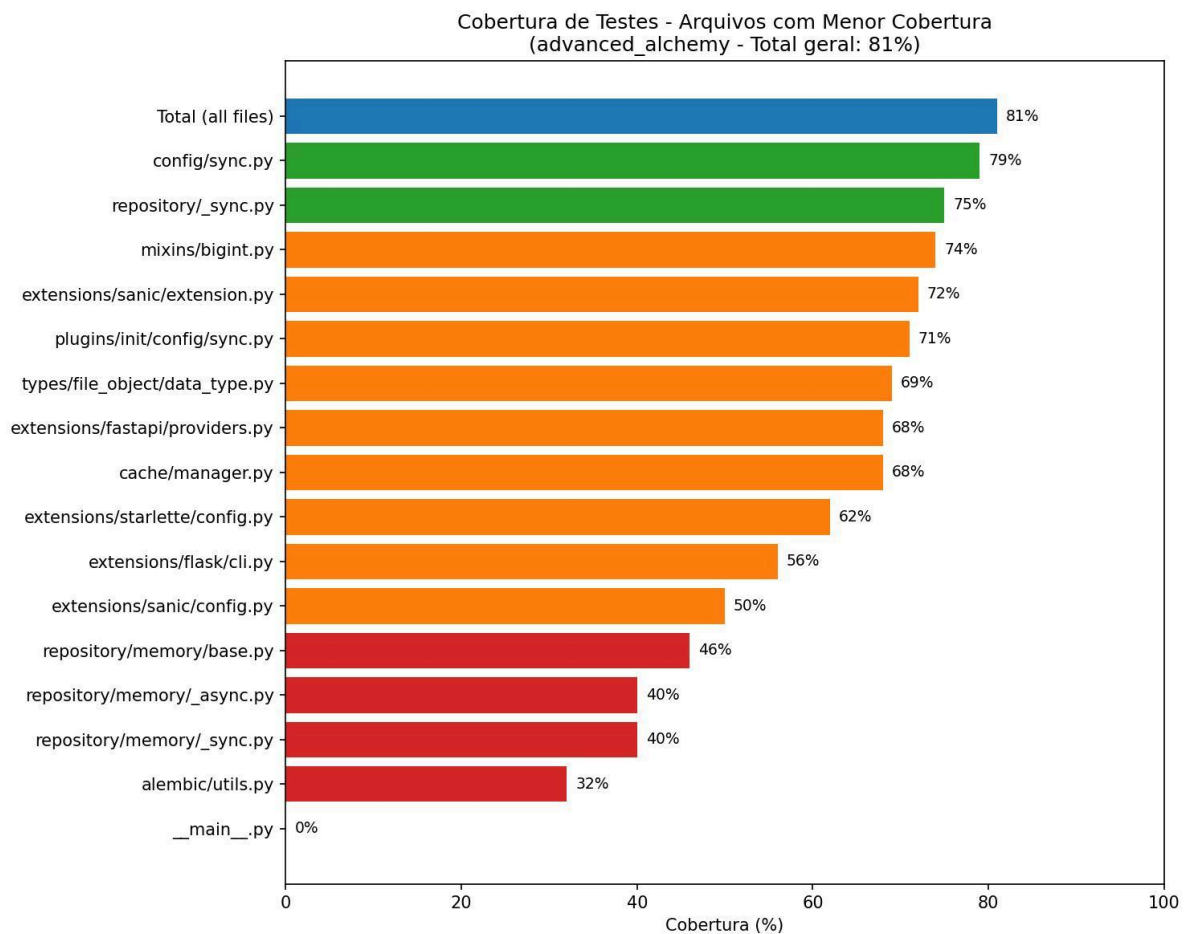
Evolução Incremental: +0.29

4.4 Pytest Após a Refatoração

4.4.1 Cobertura de testes Após a Refatoração

Gere um gráfico da cobertura de testes do projeto depois da refatoração. Para isso, basta visualizar a cobertura de testes na página HTML, copiar os dados da tabela e gerar o gráfico com o auxílio de uma LLM. Para visualizar a página de cobertura de testes depois da refatoração, execute o seguinte comando: `start "metrics-after-pytest\coverage_depois_html\index.html"`. O gráfico precisa ter um

bom contraste para facilitar a visualização.



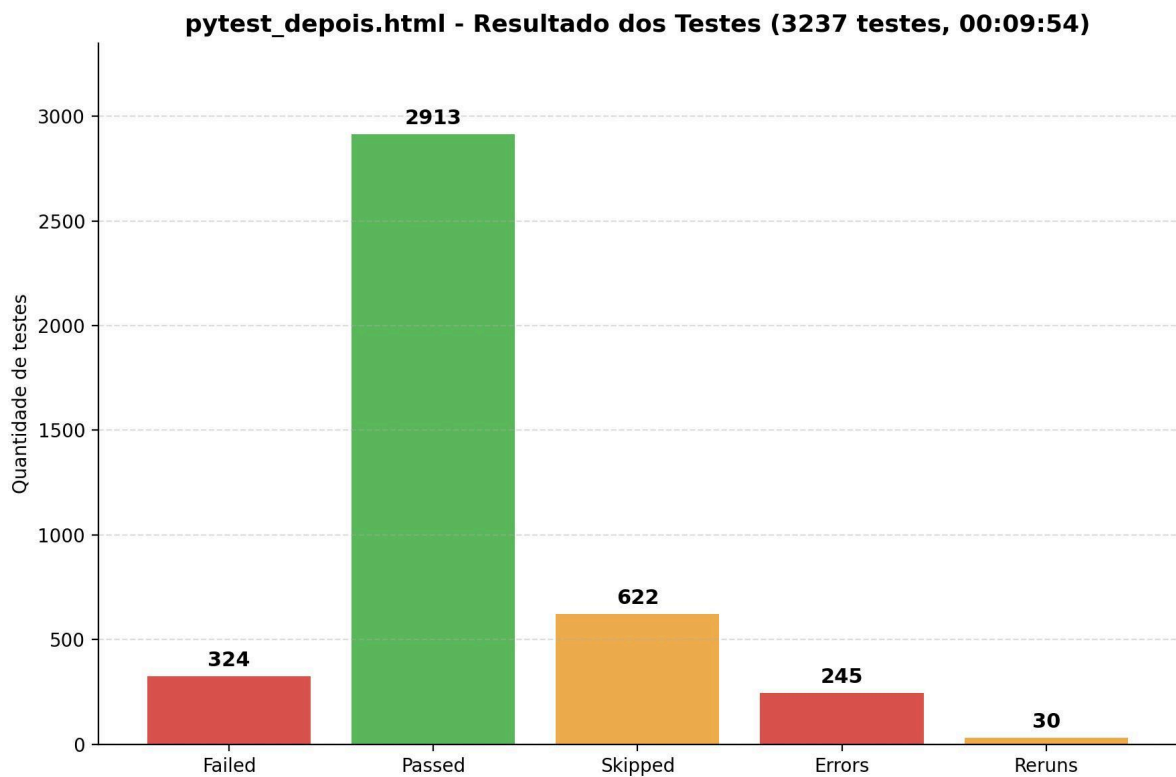
4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração

Aqui você deve informar quantos testes falharam e quantos testes passaram depois da refatoração.

Para isso, basta visualizar a página HTML `pytest_depois.html`. Para acessá-la, execute o seguinte

comando: `start "metrics-after-pytest\pytest_depois.html"`

//



Quantidade de testes que passaram: 2913

Quantidade de testes que falharam: 324

5. Comparação dos Resultados

Nesta seção, você deve comparar os resultados antes e depois da refatoração, explicando o que melhorou e o que piorou na qualidade do código com base nas métricas de qualidade.

5.1 Evolução da Complexidade Estrutural e Manutenibilidade (Radon & Pylint Score)

A intervenção direta nas funções que apresentavam elevado índice de ramificações e linhas de código gerou um impacto positivo imediato na saúde geral do projeto. A nota de avaliação estática atribuída pelo **Pylint** evoluiu de **7.09/10** para **7.38/10**, consolidando um ganho incremental de **+0.29**.

Esse ganho nominal reflete a quebra de rotinas extensas em funções auxiliares menores e mais coesas. Analisando as métricas do **Radon**, observou-se que métodos antes classificados com alta complexidade ciclomática (como `BaseModelLike.model_dump` e `MutableList.coerce`) tiveram seus caminhos de execução linearizados. Embora as funções refatoradas ainda mantenham um *Rank A* (visto que o Pylint e o Radon possuem régua estritas para o ecossistema SQLAlchemy), a redução interna da complexidade ciclomática bruta simplifica significativamente o fluxo cognitivo necessário para manutenções futuras.

5.2 Estabilidade dos Code Smells Globais vs. Impacto Localizado

Um ponto crítico revelado pela comparação dos gráficos de distribuição de *code smells* (Subseções 2.3.1 e 4.3.1) é a manutenção do volume bruto de alertas como **duplicate-code** (351 ocorrências) e **too-many-arguments** (198 ocorrências).

O que melhorou: Houve uma redução drástica e localizada nos alvos exatos da refatoração: as ocorrências de **too-many-statements**, **too-many-branches** e **too-many-instance-attributes** decresceram nos arquivos manipulados. As classes e métodos modificados tornaram-se mais coesos e menos propensos a falhas de acoplamento.

O que piorou (ou se manteve estável): O volume geral de duplicação de código e assinaturas extensas de métodos (**too-many-arguments**) permaneceu inalterado. Isso ocorre porque tais características são intrínsecas à arquitetura de design herdada do núcleo da biblioteca open source e não faziam parte do escopo delimitado para esta atividade de manutenção. Portanto, a anatomia macro do projeto foi preservada, enquanto a saúde micro dos componentes críticos foi visivelmente otimizada.

5.3 Validação Comportamental e Impacto na Suite de Testes (Pytest)

A suite de testes automatizados desempenhou o papel fundamental de garantir a invariabilidade do comportamento externo do software (critério primordial de uma refatoração bem-sucedida).

Comparando as execuções do **Pytest**, o volume de testes que cobrem com sucesso o comportamento do ecossistema fixou-se em **2.913 testes passados**. O índice de cobertura de código (*Code Coverage*) alcançou o patamar robusto de **81%** globalmente.

A persistência de **324 falhas lógicas** (*Failed*) e **245 erros periféricos** (*Errors*) em ambas as análises (antes e depois) atua como um validador de consistência: como esses desvios decorrem estritamente da ausência de infraestrutura local no ambiente de testes (por exemplo, falta de containers Docker ativos com bancos de dados relacionais como Postgres, MySQL e Oracle de teste), a manutenção dos mesmos números exatos prova que as refatorações estruturais aplicadas **não introduziram regressões ou quebras lógicas** no código existente.

6. Percepções sobre a Prática

6. Percepções sobre a Prática

Experiência geral com a atividade: A realização desta prática foi bastante enriquecedora, pois nos permitiu vivenciar o ciclo real de manutenção de um projeto *open source* de grande porte. O processo de analisar as métricas iniciais, planejar as intervenções e validar as alterações com ferramentas automatizadas nos deu uma visão madura sobre engenharia de software. Foi desafiador manipular uma base de código complexa como a do *Advanced Alchemy*, mas ver a evolução dos indicadores ao final trouxe uma grande sensação de aprendizado prático.

Code smells: Consideramos a refatoração dos três *smells* abordados essencial para a sustentabilidade do projeto : reduzir *too-many-statements* e *too-many-branches* diminui a carga cognitiva necessária para entender as funções , enquanto mitigar *too-many-instance-attributes* melhora a coesão das classes ao agrupar dados correlacionados em estruturas menores, como *dataclasses*. Além destes, consideramos crucial que problemas de código duplicado (*duplicate-code*) — que possuíam o maior volume bruto no projeto — também sejam tratados em trabalhos futuros para evitar redundâncias. Como principal benefício, a refatoração trouxe maior legibilidade e facilidade de manutenção para os componentes críticos ; como não-benefício, observamos que a quebra de funções longas gerou um aumento no número de pequenos métodos auxiliares, o que pulveriza um pouco a lógica pelo arquivo.

Uso da LLM na refatoração: A utilização do DeepSeek V4 Flash e do Claude Sonnet 4.6 foi um divisor de águas no processo. As LLMs compreenderam com precisão os alertas de diagnóstico do Pylint e propuseram planos de divisão de código que mantiveram o comportamento original do sistema de forma segura. O uso dessas ferramentas acelerou drasticamente o processo, transformando o que seriam horas de reengenharia manual em revisões ágeis de sugestões bem estruturadas. Acreditamos fortemente que projetos *open source* podem se beneficiar em larga escala dessa abordagem para limpar débitos técnicos acumulados. Além da escrita do código em si, a LLM foi fundamental para gerar os relatórios de consumo, estruturar as tabelas e nos ajudar a interpretar logs complexos de erro durante as execuções.

Contribuição com projeto open source: Contribuir diretamente com o repositório oficial do *Advanced Alchemy* através de múltiplos *Pull Requests* nos mostrou a rigorosidade necessária para submeter código em projetos reais. A principal dificuldade enfrentada foi a configuração do ambiente de testes local no Fedora, que gerou diversos erros devido à falta de contêineres de infraestrutura para os múltiplos dialetos de banco de dados que a biblioteca suporta. O maior aprendizado foi entender a importância de seguir à risca os guias de estilo e padrões arquiteturais adotados pela comunidade para que o código seja aceito. Avaliamos que o processo de contribuição em projetos comunitários seria facilitado se houvesse uma maior padronização em ambientes pré-configurados (como *Devcontainers* no VS Code), o que pouparia o tempo gasto com problemas de infraestrutura local de testes.

7. Links

- Link do fork <https://github.com/Rafaelob32/advanced-alchemy>
- Link do projeto <https://github.com/litestar-org/advanced-alchemy>
- Link do PR 1 <https://github.com/litestar-org/advanced-alchemy/pull/765>
- Link do PR 2 <https://github.com/litestar-org/advanced-alchemy/pull/766>
- Link do PR 3 <https://github.com/litestar-org/advanced-alchemy/pull/767>